

Tiny HTTP Server 升级指导书

从静态文件服务器向微型 Nginx 演进

阅读建议：这份指导书假设你已经完成了基础项目的学习（`docs/learning-roadmap.pdf`）。如果你还没有跑通过基础版 `tiny_httpd`，请先完成那 30 课再回来。

本指导书的每一个步骤都写得极其详细——**打开哪个文件、按什么键、输入什么内容、期望看到什么输出**——全部写清楚。只要你跟着一步一步做，就一定能跑通。

如果你在某一步卡住了，回头检查：文件路径对不对、分号写了没有、大括号有没有配对、`cmake --build build` 的报错信息是什么。

0. 升级前准备

0.1 确认你的起点

打开终端，逐条执行以下命令。**如果任何一步失败，先解决它再继续：**

```
# 第1步：确认你在项目根目录
cd ~/projects/tiny-http-server
pwd
# 期望输出：/home/你的用户名/projects/tiny-http-server

# 第2步：确认能编译通过
cmake -S . -B build
cmake --build build
# 期望：没有任何 Error，最后一行显示 Built target tiny_httpd

# 第3步：确认服务器能启动
./build/tiny_httpd
# 期望输出：配置：port=8080 root=www workers=4
# 看到这行后按 Ctrl+C 停止

# 第4步：用 curl 测试（需要两个终端）
# 终端1：./build/tiny_httpd
# 终端2：curl -v http://localhost:8080/
# 期望：返回 <h1>Tiny HTTP Server</h1>...

# 第5步：确认测试能通过
./build/parser_tests
# 期望输出：parser tests passed
```

0.2 当前项目结构

```
tiny-http-server/
├── CMakeLists.txt
├── include/tiny_http/
│   ├── config.h          ← 6个字段的配置结构体
│   ├── http_request.h    ← 请求结构体
│   ├── http_parser.h     ← HTTP 解析器
│   ├── http_response.h   ← HTTP 响应生成
│   ├── linux_server.h    ← 核心服务器类
│   ├── thread_pool.h     ← 线程池
│   ├── blocking_queue.h  ← 阻塞队列模板
│   ├── mime_types.h      ← MIME 类型映射
│   └── async_logger.h    ← 异步日志
├── src/
│   └── main.cpp          ← 程序入口
```

```

├── linux_server.cpp ← epoll 事件循环 + 读写处理
├── http_parser.cpp ← HTTP/1.1 解析
├── http_response.cpp ← 响应序列化
├── thread_pool.cpp ← 线程池实现
├── mime_types.cpp ← 扩展名→MIME
├── async_logger.cpp ← 后台写日志
├── tests/
│   └── parser_tests.cpp
├── www/
│   └── index.html
└── build/ ← CMake 构建产物

```

0.3 本课会用到的 vim 基础操作

如果你不熟悉 vim，这里是最小必要操作：

操作	按键
打开文件	<code>vim 文件路径</code>
进入编辑模式	按 <code>i</code>
退出编辑模式	按 <code>Esc</code>
保存并退出	按 <code>Esc</code> 然后输入 <code>:wq</code> 回车
不保存强制退出	按 <code>Esc</code> 然后输入 <code>:q!</code> 回车
显示行号	按 <code>Esc</code> 然后输入 <code>:set number</code> 回车
跳到第N行	按 <code>Esc</code> 然后输入 <code>:N</code> 回车（N是数字）
搜索文字	按 <code>Esc</code> 然后输入 <code>/关键字</code> 回车
删除当前行	按 <code>Esc</code> 然后输入 <code>dd</code>
撤销	按 <code>Esc</code> 然后输入 <code>u</code>
复制当前行	按 <code>Esc</code> 然后输入 <code>yy</code>
粘贴	按 <code>Esc</code> 然后输入 <code>p</code>

完整编辑流程示例：

1. `vim include/tiny_http/config.h` ← 打开文件
2. 按 `i` ← 进入编辑模式（左下角出现 `-- INSERT --`）
3. 用方向键移动到要修改的位置，输入代码
4. 按 `Esc` ← 退出编辑模式
5. 输入 `:wq` 回车 ← 保存并退出

如果你更习惯用 VS Code 或其他编辑器，完全可以——打开文件、编辑、保存即可。vim 不是必须的。

第一部分：五个核心课时

第 1 课：配置文件解析系统

本节目标

手写一个配置文件解析器。完成后，你的服务器可以从配置文件读取参数，命令行参数可以覆盖配置文件的值。

为什么这是第 1 课：配置文件是后续所有功能的"基础设施"——后面的第 4 课（Range）、第 5 课（sendfile）的阈值参数、第 6-17 课几乎每个新功能都需要配置项。先把配置系统做好，后面每加一个新功能只需要在配置结构体里加一个字段、在解析器里加一行指令匹配。

理论知识：为什么需要配置文件

当前方式的局限：

```
./build/tiny_httpd 8080 /var/www 8
```

参数靠位置记忆：第1个是端口，第2个是根目录，第3个是线程数。三个还行，但如果你要配置虚拟主机、代理转发、速率限制、自定义日志级别——命令行就完全不够用了。

我们要实现的配置格式（仿 Nginx 风格）：

```
# tiny_httpd.conf
server {
    listen 8080;
    root "www";
    worker_threads 4;
    keep_alive 15;
    max_events 1024;
    log_path "tiny_httpd.log";
    log_level info;
    max_body_size 1048576;

    # 虚拟主机
    server_name example.com {
        root "/var/www/example";
    }

    # 路径规则
    location /api/ {
        proxy_pass "http://localhost:5000";
    }
    location /static/ {
        root "/srv/static";
        expires 7d;
    }
}
```

```
}  
}
```

解析器架构——分两层处理：

配置文件文本



词法分析器
(Lexer)

把字符流切成 Token

"listen 8080;" → [KEYWORD, NUMBER, SEMICOLON]



语法分析器
(Parser)

把 Token 序列变成 C++ 结构体

[KEYWORD("listen"), NUMBER(8080)] → config.port = 8080



ServerConfig 结构体（你的代码使用它）

Token 类型一览：

Token	示例	说明
Keyword	listen , server , location	配置指令名
Number	8080 , 15	整数值
String	"example.com"	双引号包裹的字符串
Path	/api/	以 / 开头的路径
Semicolon	;	指令结束符
LBrace	{	块开始
RBrace	}	块结束
Eof	（文件末尾）	输入结束

涉及文件一览

本课需要修改和新建的文件：

操作	文件	说明
修改	include/tiny_http/config.h	扩展配置结构体

操作	文件	说明
新建	<code>include/tiny_http/config_parser.h</code>	解析器头文件
新建	<code>src/config_parser.cpp</code>	解析器实现（Lexer + Parser + 接口）
修改	<code>src/main.cpp</code>	集成配置文件加载
修改	<code>CMakeLists.txt</code>	添加新文件到构建系统
新建	<code>tiny_httpd.conf</code>	示例配置文件
新建	<code>tests/config_parser_tests.cpp</code>	配置解析器测试

步骤 1：扩展配置结构体（config.h）

要做什么：在现有的 `ServerConfig` 结构体中增加新字段，并在它前面新增两个辅助结构体。

1.1 打开文件

打开终端，输入：

```
vim include/tiny_http/config.h
```

你看到的当前内容如下（这就是升级前的原貌）：

```
#pragma once

#include <cstdint>
#include <string>

namespace tiny_http {

struct ServerConfig {
    int port {8080};
    std::string document_root {"www"};
    std::size_t worker_threads {4};
    int keep_alive_seconds {15};
    int max_events {1024};
    std::string log_path {"tiny_httpd.log"};
};

} // namespace tiny_http
```

1.2 操作步骤

第1小步：按 `i` 进入编辑模式。

第2小步：用方向键把光标移到 `#include <string>` 这行的末尾，按回车换行，输入：

```
#include <vector>
```

改完后这一片看起来像：

```
#include <cstdint>
#include <string>
#include <vector>
```

为什么要加 `<vector>`？因为后面要用 `std::vector<VirtualHost>` 和 `std::vector<LocationRule>` 存多个虚拟主机和 location 规则。

第3小步：把光标移到 `struct ServerConfig {` 这一行的上面（也就是 `namespace tiny_http {` 的下面），按 `i` 确保在编辑模式，输入以下两个新结构体：

```
// ---- 一个 location 规则 ----
// 例如: location /api/ { proxy_pass http://localhost:5000; }
struct LocationRule {
    std::string path;           // 匹配路径, 如 "/api/"
    std::string root;          // 可选: 替代 document_root
    std::string proxy_pass;     // 可选: 反向代理目标 URL
    std::string expires;       // 可选: 缓存过期时间, 如 "7d"、"1h"
};

// ---- 一个虚拟主机 ----
// server_name "example.com" { root /var/www/example; }
struct VirtualHost {
    std::string server_name;    // 域名
    std::string root;          // 该虚拟主机的根目录
};
```

逐行解释：

- `struct LocationRule`：每个 `location /xxx/ { ... }` 块解析后生成一个 `LocationRule` 对象。四个字段都是 `std::string`，空字符串表示“未设置”。后面 `server` 代码用 `if (!loc.proxy_pass.empty())` 判断是否配置了代理。
- `struct VirtualHost`：每个 `server_name "xxx" { ... }` 块解析后生成一个。字段很简单——域名 + 对应的根目录。

第4小步：在 `ServerConfig` 结构体内部，原有 6 个字段的下面（`log_path` 那行之后），新增以下 6 行：

```
// ---- 新增字段 ----
std::string log_level {"info"};           // debug / info / warn / error
std::string config_file;                  // 配置文件路径（记录用）
std::vector<VirtualHost> virtual_hosts;    // 虚拟主机列表
std::vector<LocationRule> locations;      // location 规则列表
```

```
std::size_t max_body_size {1048576};           // 最大请求体大小（默认 1MB）
int rate_limit {0};                             // 每秒最大请求数（0 = 不限制）
```

逐字段解释：

- `log_level`：控制日志输出级别。默认 `"info"`，开发调试时可以改为 `"debug"`。
- `config_file`：记录当前使用的配置文件路径，方便调试时确认"我到底加载了哪个配置"。
- `virtual_hosts`：虚拟主机列表。当请求的 `Host` 头匹配某个虚拟主机时，使用该主机的 `root` 替代默认 `document_root`。
- `locations`：location 规则列表。按顺序匹配请求路径，第一个匹配的规则生效。
- `max_body_size`：限制 POST/PUT 请求体大小，默认 1048576 字节（1MB）。防止恶意大请求耗尽内存。
- `rate_limit`：简单的速率限制。0 表示不限制。设置为 100 就表示每秒最多 100 个请求。

1.3 保存并验证

按 `Esc`，输入 `:wq` 回车，保存退出。

验证编译：

```
cmake --build build
```

期望输出：编译成功，没有错误。虽然新字段还没被使用（会有 `unused-field` 警告，可以忽略），但语法必须正确。

1.4 最终文件内容（确认你改对了）

你改完后的 `config.h` 应该像这样（可以 `cat include/tiny_http/config.h` 确认）：

```
#pragma once

#include <cstdint>
#include <string>
#include <vector>

namespace tiny_http {

    struct LocationRule {
        std::string path;
        std::string root;
        std::string proxy_pass;
        std::string expires;
    };

    struct VirtualHost {
        std::string server_name;
        std::string root;
    };

}
```

```

struct ServerConfig {
    int port {8080};
    std::string document_root {"www"};
    std::size_t worker_threads {4};
    int keep_alive_seconds {15};
    int max_events {1024};
    std::string log_path {"tiny_httpd.log"};

    // ---- 新增 ----
    std::string log_level {"info"};
    std::string config_file;
    std::vector<VirtualHost> virtual_hosts;
    std::vector<LocationRule> locations;
    std::size_t max_body_size {1048576};
    int rate_limit {0};
};

} // namespace tiny_http

```

到这里，步骤1完成。如果 `cmake --build build` 有报错，停下来检查：是不是少写了分号？大括号配对了没？`#include <vector>` 加了吗？

步骤 2：创建配置解析器头文件（config_parser.h）

要做什么：新建一个头文件，声明两个公开函数。

2.1 创建文件

在终端输入：

```

touch include/tiny_http/config_parser.h
vim include/tiny_http/config_parser.h

```

2.2 输入内容

按 `i` 进入编辑模式，输入以下全部内容：

```

#pragma once

#include "tiny_http/config.h"
#include <string>

namespace tiny_http {

    // 解析配置文件，返回填充好的 ServerConfig
    // 解析失败时抛出 std::runtime_error（包含行号和错误描述）
    ServerConfig parse_config_file(const std::string& file_path);
}

```

```
// 将命令行参数覆盖到 config 上（命令行优先级更高）
// 用法: config.port 默认 8080, 但 ./tiny_httpd 9090 会覆盖为 9090
void apply_command_line(ServerConfig& config, int argc, char* argv[]);

} // namespace tiny_http
```

逐行解释：

- `#pragma once`：防止头文件被重复包含。如果你在多个 `.cpp` 里 `#include` 了这个头文件，编译器只会处理一次。
- `#include "tiny_http/config.h"`：引用我们刚刚修改的配置结构体定义。
- `#include <string>`：`std::string` 类型需要这个头文件。
- `parse_config_file`：接收配置文件路径，返回填充好的 `ServerConfig`。如果文件不存在或格式错误，**抛出异常**（不是返回错误码）。调用方 `catch` 后可以降级使用默认配置。
- `apply_command_line`：接收 `main()` 的 `argc / argv`，把位置参数覆盖到 `config` 上。注意 `argv` 是 `char*[]` 而不是 `const char*[]`，因为 C++ 标准规定 `main()` 的签名就是如此。

2.3 保存

按 `Esc`，输入 `:wq` 回车。

验证：这个头文件本身不能编译（没有 `.cpp` 实现），所以继续下一步——创建实现文件。

步骤 3：创建实现文件的开头部分（`config_parser.cpp`）

要做什么：新建 `src/config_parser.cpp`，写入 `include` 语句和 `Token` 定义。

3.1 创建并打开文件

```
touch src/config_parser.cpp
vim src/config_parser.cpp
```

3.2 输入文件头部和 `Token` 定义

按 `i` 进入编辑模式，输入：

```
#include "tiny_http/config_parser.h"

#include <cctype>
#include <cstdlib>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <string>
```

```

#include <vector>

namespace tiny_http {

// =====
// 第1部分: Token 定义 — 词法分析的基本单位
// =====

// Token 的类型: 配置文件由这些基本元素组成
enum class TokenType {
    Keyword,    // 如 listen, server, location, root
    Number,     // 如 8080, 15
    String,     // 如 "example.com" (双引号包裹)
    Path,       // 如 /api/ (以 / 开头)
    Semicolon,  // ;
    LBrace,     // {
    RBrace,     // }
    Eof         // 文件结束
};

// 一个 Token 对象: 类型 + 内容 + 所在行号
struct Token {
    TokenType type;
    std::string value;
    int line {0};      // 行号, 用于报错时定位
};

```

逐行解释:

- `#include "tiny_http/config_parser.h"`: 引用我们自己写的头文件。
- `#include <cctype>`: 提供 `std::isdigit()`、`std::isspace()` 等字符分类函数。
- `#include <cstdlib>`: 提供 `std::atoi()`、`std::stoi()` 等字符串转数字函数。
- `#include <fstream>`: 提供 `std::ifstream` (文件输入流), 用于读取配置文件。
- `#include <sstream>`: 提供 `std::ostringstream` (字符串输出流), 用于把整个文件读成字符串。
- `#include <stdexcept>`: 提供 `std::runtime_error`, 解析错误时抛这个异常。
- `enum class TokenType`: 用 `enum class` 而不是普通 `enum`, 因为 `enum class` 不会和整数混用, 更安全。里面定义了 8 种 Token 类型。
- `struct Token`: 每个 Token 包含三样东西——类型 (是什么)、值 (内容是什么)、行号 (在哪一行, 报错时用)。

3.3 先不保存退出? 不, 我们接着写 Lexer。

继续往下翻, 进入步骤 4。

步骤 4：在同一个文件中实现 Lexer（词法分析器）

要做什么：紧接在 Token 定义后面，在同一个 `config_parser.cpp` 中写入 Lexer 类。

4.1 确保你还在编辑模式

如果你刚才退出了 vim，重新打开：

```
vim src/config_parser.cpp
```

光标移到文件末尾（按 `G` 跳到末尾），按 `i` 进入编辑模式，继续输入：

```
// =====
// 第2部分：词法分析器 (Lexer)
// 职责：把原始字符串切成 Token 序列
// =====

class Lexer {
public:
    // 构造函数：接收整个配置文件的文本内容
    explicit Lexer(std::string input)
        : input_(std::move(input)), pos_(0), line_(1) {}

    // 读取下一个 Token（调用一次返回一个）
    Token next() {
        skip_whitespace_and_comments();    // 跳过空白和注释

        // 文件结束了
        if (pos_ >= input_.size()) {
            return {TokenType::Eof, "", line_};
        }

        char c = input_[pos_];

        // ---- 单字符 Token: 直接返回 ----
        if (c == ';') {
            pos_++;
            return {TokenType::Semicolon, ";", line_};
        }
        if (c == '{') {
            pos_++;
            return {TokenType::LBrace, "{", line_};
        }
        if (c == '}') {
            pos_++;
            return {TokenType::RBrace, "}", line_};
        }

        // ---- 复杂 Token: 交给专门的扫描函数 ----
        if (c == '"') {
            return scan_string();           // "example.com"
        }
        if (std::isdigit(static_cast<unsigned char>(c))) {
            return scan_number();           // 8080
        }
    }
};
```

```

    }
    if (c == '/') {
        return scan_path();          // /api/
    }

    // 剩下的就是关键字 (如 listen, server)
    return scan_keyword();
}

private:
    // ===== 辅助函数 =====

    // 跳过空格、制表符、换行、注释 (# 开头到行尾)
    void skip_whitespace_and_comments() {
        while (pos_ < input_.size()) {
            char c = input_[pos_];

            if (c == '\n') {
                line_++;              // 遇到换行, 行号加1
                pos_++;
            } else if (c == '#') {
                // 遇到 # 号, 跳过整行 (注释)
                while (pos_ < input_.size() && input_[pos_] != '\n') {
                    pos_++;
                }
                // 注意: 不在此处 line_++, 因为换行符 \n 会在下一轮处理
            } else if (std::isspace(static_cast<unsigned char>(c))) {
                pos_++;              // 跳过普通空白
            } else {
                break;               // 遇到有效字符, 停止
            }
        }
    }

    // 扫描引号包裹的字符串: "hello world" → hello world
    Token scan_string() {
        pos_++; // 跳过开始的 "
        std::string value;
        while (pos_ < input_.size() && input_[pos_] != '"') {
            // 简单处理转义: \" 表示引号本身
            if (input_[pos_] == '\\' && pos_ + 1 < input_.size()) {
                pos_++;
            }
            value += input_[pos_];
            pos_++;
        }
        if (pos_ < input_.size()) {
            pos_++; // 跳过结束的 "
        }
        return {TokenType::String, value, line_};
    }

    // 扫描数字: 8080 → "8080"
    Token scan_number() {
        std::string value;
        while (pos_ < input_.size()
            && std::isdigit(static_cast<unsigned char>(input_[pos_]))) {
            value += input_[pos_];

```

```

        pos_++;
    }
    return {TokenType::Number, value, line_};
}

// 扫描路径: /api/v1/users → "/api/v1/users"
Token scan_path() {
    std::string value;
    while (pos_ < input_.size()
           && !std::isspace(static_cast<unsigned char>(input_[pos_]))
           && input_[pos_] != ';'
           && input_[pos_] != '{') {
        value += input_[pos_];
        pos_++;
    }
    return {TokenType::Path, value, line_};
}

// 扫描关键字/标识符: listen → "listen"
Token scan_keyword() {
    std::string value;
    while (pos_ < input_.size()
           && !std::isspace(static_cast<unsigned char>(input_[pos_]))
           && input_[pos_] != ';'
           && input_[pos_] != '{'
           && input_[pos_] != '}') {
        value += input_[pos_];
        pos_++;
    }
    return {TokenType::Keyword, value, line_};
}

// ===== 成员变量 =====
std::string input_;    // 整个配置文件的内容
size_t pos_;          // 当前读取位置（索引）
int line_;             // 当前行号（用于错误提示）
};

```

逐段解释：

- **构造函数**：接收 `std::string` 参数，用 `std::move` 转移所有权（避免拷贝）。`pos_` 初始化为 0（从第一个字符开始读），`line_` 初始化为 1（第1行）。
- **next()** 是 Lexer 的唯一公开方法。每次调用返回下一个 Token。流程：
 - 先调用 `skip_whitespace_and_comments()` 跳过空白和注释
 - 如果已经读到文件末尾，返回 `Eof` 类型的 Token
 - 看当前字符是什么：`;``{}` 直接返回对应的单字符 Token
 - `"` 开头 → `scan_string()`，数字开头 → `scan_number()`，`/` 开头 → `scan_path()`，其余 → `scan_keyword()`

- **skip_whitespace_and_comments()**: 每次取 Token 前调用。注意 `#` 的处理——跳过 `#` 到行尾的所有字符，但不在这里 `line_++`，因为换行符 `\n` 本身还在输入流里，下一轮循环会处理它并正确增加行号。
- **scan_string()**: 遇到 `"` 就一直读到下一个 `"`。简单处理了 `\` 转义（遇到反斜杠就把下一个字符当作普通字符，不当作结束引号）。
- **scan_number()**: 连续读数字字符。注意这里只支持整数，对于配置文件完全够用。存为字符串，后面用 `std::stoi()` 转成整数。
- **scan_path()**: 以 `/` 开头，读到空格、`;` 或 `{` 为止。路径里可以有字母、数字、斜杠、点、下划线。
- **scan_keyword()**: 读连续非空白字符，直到遇到分隔符。`listen`、`server`、`location`、`server_name` 等都会走到这里。

4.2 保存当前进度

按 `Esc`，输入 `:w` 回车（只保存不退出）。然后继续编辑，我们接着写 Parser。

步骤 5：在同一个文件中实现 Parser（语法分析器）

要做什么：紧接在 Lexer 类后面，写入 Parser 类。采用递归下降算法。

5.1 确保你在文件末尾

在 vim 中按 `G` 跳到文件末尾，确认光标在 Lexer 类的 `};` 之后。

按 `i` 进入编辑模式，继续输入：

```
// =====
// 第3部分：语法分析器 (Parser)
// 职责：把 Token 序列转换成 ServerConfig 结构体
// 采用"递归下降"算法——每个语法结构对应一个解析函数
// =====

class Parser {
public:
    explicit Parser(std::string input)
        : lexer_(std::move(input))
    {
        current_ = lexer_.next(); // 预读第一个 Token
    }

    // 入口：解析整个配置文件
    ServerConfig parse() {
        ServerConfig config;
```

```

        // 循环解析顶层结构，直到文件结束
        while (current_.type != TokenType::Eof) {
            parse_top_level(config);
        }

        return config;
    }

private:
    // ===== 顶层分发 =====

    // 顶层只能出现两种东西：
    // 1. server { ... } 块
    // 2. 独立指令（如 listen 8080;）
    void parse_top_level(ServerConfig& config) {
        if (current_.type == TokenType::Keyword
            && current_.value == "server") {
            parse_server_block(config);
        } else if (current_.type == TokenType::Keyword) {
            parse_directive(config);
        } else {
            error("顶层只能出现 server 块或指令，但遇到了： " + current_.value);
        }
    }

    // ===== 块结构 =====

    // server { ... } 块
    // 内部可以包含：server_name 块、location 块、普通指令
    void parse_server_block(ServerConfig& config) {
        expect(TokenType::Keyword);    // 消费 "server"
        expect(TokenType::LBrace);    // 消费 "{"

        // 循环处理块内部，直到遇到 }
        while (current_.type != TokenType::RBrace
            && current_.type != TokenType::Eof) {

            if (current_.type == TokenType::Keyword
                && current_.value == "server_name") {
                parse_virtual_host(config);
            } else if (current_.type == TokenType::Keyword
                && current_.value == "location") {
                parse_location(config);
            } else if (current_.type == TokenType::Keyword) {
                parse_directive(config);
            } else {
                error("server 块内出现了意外的 token： " + current_.value);
            }
        }

        expect(TokenType::RBrace);    // 消费 "}"
    }

    // server_name "example.com" { root "/var/www"; }
    void parse_virtual_host(ServerConfig& config) {
        expect(TokenType::Keyword);    // 消费 "server_name"
    }

```

```

VirtualHost vh;
vh.server_name = current_.value;
// 域名可以是 String 或 Keyword (不带引号的也接受)
if (current_.type == TokenType::String) {
    expect(TokenType::String);
} else {
    expect(TokenType::Keyword);
}

expect(TokenType::LBrace);    // 消费 "{"

// 处理块内部的指令
while (current_.type != TokenType::RBrace
    && current_.type != TokenType::Eof) {
    if (current_.type == TokenType::Keyword
        && current_.value == "root") {
        expect(TokenType::Keyword);    // 消费 "root"
        vh.root = current_.value;
        // root 的值可能是 String 或 Path
        if (current_.type == TokenType::String) {
            expect(TokenType::String);
        } else if (current_.type == TokenType::Path) {
            expect(TokenType::Path);
        } else {
            error("root 指令后需要一个路径或字符串");
        }
        expect(TokenType::Semicolon);
    } else {
        error("server_name 块内不支持的指令: " + current_.value);
    }
}

expect(TokenType::RBrace);    // 消费 "}"
config.virtual_hosts.push_back(std::move(vh));
}

// location /path/ { ... }
void parse_location(ServerConfig& config) {
    expect(TokenType::Keyword);    // 消费 "location"

    LocationRule loc;
    loc.path = current_.value;    // 保存路径, 如 "/api/"
    if (current_.type == TokenType::Path) {
        expect(TokenType::Path);
    } else if (current_.type == TokenType::String) {
        expect(TokenType::String);
    } else {
        error("location 后需要一个路径");
    }

    expect(TokenType::LBrace);    // 消费 "{"

    // 处理 location 块内部的指令
    while (current_.type != TokenType::RBrace
        && current_.type != TokenType::Eof) {

        if (current_.type == TokenType::Keyword
            && current_.value == "root") {

```

```

        expect(TokenType::Keyword);    // 消费 "root"
        loc.root = current_.value;
        if (current_.type == TokenType::String) {
            expect(TokenType::String);
        } else if (current_.type == TokenType::Path) {
            expect(TokenType::Path);
        } else {
            error("root 指令后需要一个路径");
        }
        expect(TokenType::Semicolon);

    } else if (current_.type == TokenType::Keyword
        && current_.value == "proxy_pass") {
        expect(TokenType::Keyword);    // 消费 "proxy_pass"
        loc.proxy_pass = current_.value;
        if (current_.type == TokenType::String) {
            expect(TokenType::String);
        } else {
            expect(TokenType::Keyword); // 不带引号的 URL
        }
        expect(TokenType::Semicolon);

    } else if (current_.type == TokenType::Keyword
        && current_.value == "expires") {
        expect(TokenType::Keyword);    // 消费 "expires"
        loc.expires = current_.value;
        expect(TokenType::Keyword);    // 如 "7d", "1h"
        expect(TokenType::Semicolon);

    } else {
        error("location 块内不支持的指令: " + current_.value);
    }
}

expect(TokenType::RBrace);    // 消费 "}"
config.locations.push_back(std::move(loc));
}

// ===== 独立指令 =====

// 解析一条指令, 如: listen 8080;
// 根据指令名分发到对应的字段
void parse_directive(ServerConfig& config) {
    std::string name = current_.value;
    expect(TokenType::Keyword);    // 消费指令名

    if (name == "listen") {
        config.port = std::stoi(current_.value);
        expect(TokenType::Number);

    } else if (name == "root") {
        config.document_root = current_.value;
        if (current_.type == TokenType::String) {
            expect(TokenType::String);
        } else if (current_.type == TokenType::Path) {
            expect(TokenType::Path);
        } else {
            error("root 需要一个路径参数");
        }
    }
}

```

```

    }

    } else if (name == "worker_threads") {
        config.worker_threads
            = static_cast<std::size_t>(std::stoul(current_.value));
        expect(TokenType::Number);

    } else if (name == "keep_alive") {
        config.keep_alive_seconds = std::stoi(current_.value);
        expect(TokenType::Number);

    } else if (name == "max_events") {
        config.max_events = std::stoi(current_.value);
        expect(TokenType::Number);

    } else if (name == "log_path") {
        config.log_path = current_.value;
        if (current_.type == TokenType::String) {
            expect(TokenType::String);
        } else {
            expect(TokenType::Keyword); // 无引号的路径也接受
        }

    } else if (name == "log_level") {
        config.log_level = current_.value;
        expect(TokenType::Keyword);

    } else if (name == "max_body_size") {
        config.max_body_size
            = static_cast<std::size_t>(std::stoul(current_.value));
        expect(TokenType::Number);

    } else if (name == "rate_limit") {
        config.rate_limit = std::stoi(current_.value);
        expect(TokenType::Number);

    } else {
        // 遇到不认识的指令，报错（不默默忽略，帮助发现拼写错误）
        error("未知的配置指令: " + name);
    }

    // 每条指令必须以分号结尾
    expect(TokenType::Semicolon);
}

// ===== 辅助函数 =====

// 断言当前 Token 的类型，匹配则消费并读取下一个
// 不匹配则抛出异常
void expect(TokenType type) {
    if (current_.type != type) {
        error("期望 " + token_type_name(type)
            + ", 但遇到了: " + current_.value);
    }
    current_ = lexer_.next(); // 前进到下一个 Token
}

// 拼一个人类可读的 Token 类型名（用于错误提示）

```

```

static std::string token_type_name(TokenType t) {
    switch (t) {
        case TokenType::Keyword:    return "关键字";
        case TokenType::Number:     return "数字";
        case TokenType::String:     return "字符串";
        case TokenType::Path:       return "路径";
        case TokenType::Semicolon:  return "';'";
        case TokenType::LBrace:     return "'{'";
        case TokenType::RBrace:     return "'}'";
        case TokenType::Eof:        return "文件结尾";
        default:                    return "未知";
    }
}

// 报错: 抛出包含行号的异常
[[noreturn]] void error(const std::string& msg) {
    throw std::runtime_error(
        "配置文件第 " + std::to_string(current_.line) + " 行: " + msg);
}

// ===== 成员变量 =====
Lexer lexer_;           // 词法分析器
Token current_;         // 当前正在处理的 Token (lookahead)
};

```

逐段解释:

- **parse():** 入口函数。循环调用 `parse_top_level()` 直到文件结束 (`Eof`)。返回填充好的 `ServerConfig`。注意这里每次调用 `parse_top_level()` 可能会消耗不同数量的 token——可能是一条指令, 也可能是一个完整的嵌套块。
- **parse_top_level():** 顶层分发。看当前 token:
 - 如果是 `Keyword` 且值是 `"server"` → 这是一个 `server` 块的开始, 交给 `parse_server_block()`
 - 如果是其他 `Keyword` → 这是一条独立指令 (如 `listen 8080;`), 交给 `parse_directive()`
 - 其他都算错误
- **parse_server_block():** `server` 关键字 → `{` → 循环处理内部 (可能是 `server_name` 子块、`location` 子块或普通指令) → `}`。这就是"递归下降"的体现——`server` 块内部可以递归包含其他块。
- **parse_virtual_host():** `server_name "domain" { root "..."; }`。注意域名支持带引号和不带引号两种写法 (通过 `current_.type` 判断)。解析完成后 `push_back` 到 `config.virtual_hosts`。

- **parse_location()**: `location /path/ { root "..."; proxy_pass "..."; expires 7d; }`。三个子指令都是可选的——用 `if-else` 逐个判断。解析完成后 `push_back` 到 `config.locations`。
- **parse_directive()**: 通用指令解析。先拿到指令名 (`name = current_.value`)，然后 `if-else` 分发到对应的配置字段。**遇到不认识的指令会报错**——不默默忽略，帮助发现拼写错误。每条指令最后必须用 `;` 结尾。
- **expect()**: 这是整个解析器的**核心原语**。断言当前 `token` 的类型，匹配就消费掉（读下一个），不匹配就报错。所有解析逻辑都通过反复调用 `expect()` 来推进 `token` 流。例如 `expect(TokenType::Semicolon)` 的意思是“我期望下一个 `token` 是分号，如果不是就报错”。
- **token_type_name()**: 把一个 `TokenType` 枚举值转成人类可读的中文字符串。只在报错时用到——让错误信息显示“期望 '`;`'，但遇到了: 8080”而不是“期望 4，但遇到了: 8080”。
- **error()**: 报错函数。 `[[noreturn]]` 是 C++11 的属性，告诉编译器“这个函数绝对不会正常返回”（因为它总是抛异常）。这有两个好处：消除“函数没有 `return` 语句”的编译警告，帮助编译器做优化。

递归下降的核心思想：每个语法结构（`server` 块、`location` 块、指令）对应一个函数，函数之间可以相互调用。这样的代码结构和语法的树状结构一一对应，直观易读。当你需要支持新的语法结构时，只需要加一个新的 `parse_xxx()` 函数。

5.2 保存当前进度

按 `Esc`，输入 `:w` 回车（只保存不退出）。然后继续编辑，我们接着写公开接口。

步骤 6：在 `config_parser.cpp` 末尾实现公开接口

要做什么：紧接在 `Parser` 类后面，写入两个公开函数。

6.1 确保你在文件末尾

在 `vim` 中按 `G` 跳到文件末尾，按 `i` 进入编辑模式，继续输入：

```
// =====
// 第4部分：公开接口
// =====

// 从文件路径加载并解析配置
ServerConfig parse_config_file(const std::string& file_path) {
    // 1. 打开文件
    std::ifstream file(file_path);
```

```

    if (!file) {
        throw std::runtime_error("无法打开配置文件: " + file_path);
    }

    // 2. 读取整个文件到字符串
    std::ostringstream oss;
    oss << file.rdbuf();
    std::string content = oss.str();

    // 3. 检查空文件
    if (content.empty()) {
        // 空文件 = 使用默认配置, 不报错
        return ServerConfig{};
    }

    // 4. 交给 Parser 解析
    Parser parser(std::move(content));
    ServerConfig config = parser.parse();
    config.config_file = file_path;    // 记录配置文件路径
    return config;
}

// 命令行参数覆盖配置值
void apply_command_line(ServerConfig& config, int argc, char* argv[]) {
    // 位置参数: [port] [document_root] [worker_threads]
    if (argc >= 2) {
        config.port = std::atoi(argv[1]);
    }
    if (argc >= 3) {
        config.document_root = argv[2];
    }
    if (argc >= 4) {
        config.worker_threads
            = static_cast<std::size_t>(std::atoi(argv[3]));
    }
}

} // namespace tiny_http

```

逐行解释：

- `parse_config_file()` 的流程：
- 用 `std::ifstream` 打开文件。如果 `!file` 为真（打开失败），直接抛异常。
- `oss << file.rdbuf()` 把整个文件内容读进 `std::ostringstream`，然后 `.str()` 取出 `std::string`。这是 C++ 中读整个文件的标准写法。
- 空文件不报错——返回一个全默认的 `ServerConfig{}`。这是为了健壮性：假设用户 `touch tiny_httpd.conf` 创建了一个空文件，服务器应该用默认配置正常启动，而不是崩溃。
- 把内容 `std::move` 给 Parser（转移所有权，避免拷贝大字符串），调用 `parser.parse()` 得到配置。

- `apply_command_line()`：从 `argv[1]`、`argv[2]`、`argv[3]` 分别覆盖 `port`、`root`、`worker_threads`。注意这几个参数是可选的——`if (argc >= N)` 确保只处理实际传入的参数。保持与旧版本完全兼容。

6.2 保存文件

按 `Esc`，输入 `:wq` 回车，保存并退出。

验证：现在尝试编译，确保 `Lexer`、`Parser`、公开接口都没有语法错误：

```
cmake --build build
```

如果有编译错误：

- 检查大括号是否配对
- 检查分号是否遗漏
- 检查 `namespace tiny_http {` 和结尾的 `}` 是否匹配
- 检查 `TokenType::Eof` 是否拼写正确（注意大小写）

期望输出：编译成功。可能有一些 "unused parameter" 警告，可以忽略。

步骤 7：修改 `main.cpp` 集成配置文件加载

要做什么：修改 `src/main.cpp`，让程序启动时先加载配置文件，再用命令行覆盖。

7.1 打开文件

```
vim src/main.cpp
```

你看到的当前内容（升级前）：

```
#include "tiny_http/linux_server.h"

#include <csignal>
#include <iostream>
#include <cstdlib>

// 全局服务器指针（信号处理函数需要访问）
tiny_http::LinuxHttpServer* g_server = nullptr;

// 信号处理函数
void handle_signal(int /*signum*/)
{
    if (g_server != nullptr) {
        g_server->stop(); // 触发优雅关闭
    }
}
```

```

    }
}

int main(int argc, char* argv[])
{
    // 注册信号处理
    std::signal(SIGINT, handle_signal); // Ctrl+C
    std::signal(SIGTERM, handle_signal); // kill 命令

    tiny_http::ServerConfig config;

    // 命令行参数: [port] [document_root] [worker_threads]
    if (argc >= 2) {
        config.port = std::atoi(argv[1]);
    }
    if (argc >= 3) {
        config.document_root = argv[2];
    }
    if (argc >= 4) {
        config.worker_threads = static_cast<std::size_t>(std::atoi(argv[3]));
    }

    std::cout << "配置: port=" << config.port
                << " root=" << config.document_root
                << " workers=" << config.worker_threads << "\n";

    tiny_http::LinuxHttpServer server(config);
    g_server = &server;

    if (!server.start()) {
        std::cerr << "启动失败! \n";
        return 1;
    }

    std::cout << "服务器已正常退出\n";
    return 0;
}

```

7.2 需要做的修改

修改1: 在文件顶部的 `#include` 区域, 新增两行:

在 `#include <cstdlib>` 下面加一行:

```
#include <string>
```

在 `#include "tiny_http/linux_server.h"` 下面加一行:

```
#include "tiny_http/config_parser.h"
```

修改2: 删除原来命令行参数解析的那几行 (从 `// 命令行参数` 到 `config.worker_threads = ...`), 替换为"先加载配置文件、再命令行覆盖"的逻辑。

按 `i` 进入编辑模式，找到 `tiny_http::ServerConfig config;` 这一行之后的内容，把从 `// 命令行参数` 开始一直到 `std::cout << "配置: ..."` 之前的内容**全部删除**，替换为以下代码：

```
// =====
// 第1步：尝试加载配置文件
// =====

// 默认配置文件名
std::string config_path = "tiny_httpd.conf";

// 检查命令行中是否通过 -c 指定了配置文件
for (int i = 1; i < argc; ++i) {
    if (std::string(argv[i]) == "-c" && i + 1 < argc) {
        config_path = argv[i + 1];
        break;
    }
}

try {
    config = tiny_http::parse_config_file(config_path);
    std::cout << "[配置] 已加载配置文件: " << config_path << "\n";
} catch (const std::exception& e) {
    // 配置文件加载失败不致命—打印警告，使用默认配置继续
    std::cerr << "[配置] 警告: " << e.what() << "\n";
    std::cerr << "[配置] 将使用默认配置启动\n";
}

// =====
// 第2步：命令行参数覆盖（优先级高于配置文件）
// =====
tiny_http::apply_command_line(config, argc, argv);

// =====
// 第3步：打印最终生效的配置
// =====
std::cout << "=====\n";
std::cout << "Tiny HTTP Server 启动配置:\n";
std::cout << "  port          = " << config.port << "\n";
std::cout << "  document_root = " << config.document_root << "\n";
std::cout << "  worker_threads = " << config.worker_threads << "\n";
std::cout << "  keep_alive     = " << config.keep_alive_seconds << "s\n";
std::cout << "  max_events     = " << config.max_events << "\n";
std::cout << "  log_path       = " << config.log_path << "\n";
std::cout << "  log_level      = " << config.log_level << "\n";
std::cout << "  max_body_size  = " << config.max_body_size << " bytes\n";
if (config.rate_limit > 0) {
    std::cout << "  rate_limit     = " << config.rate_limit << " req/s\n";
}
if (!config.virtual_hosts.empty()) {
    std::cout << "  virtual_hosts = "
        << config.virtual_hosts.size() << " 个\n";
}
if (!config.locations.empty()) {
    std::cout << "  locations     = "
        << config.locations.size() << " 个\n";
}
```

```
}
std::cout << "=====\n";
```

修改3： `std::cout << "配置: port=" << config.port ...` 这一行需要删除——我们已经有更详细的新版配置打印了。

7.3 最终 main.cpp 全貌

修改完成后，你的 `src/main.cpp` 应该像这样（可以 `cat src/main.cpp` 确认）：

```
#include "tiny_http/linux_server.h"
#include "tiny_http/config_parser.h"

#include <csignal>
#include <iostream>
#include <cstdlib>
#include <string>

// 全局服务器指针（信号处理函数需要访问）
tiny_http::LinuxHttpServer* g_server = nullptr;

// 信号处理函数
void handle_signal(int /*signum*/)
{
    if (g_server != nullptr) {
        g_server->stop(); // 触发优雅关闭
    }
}

int main(int argc, char* argv[])
{
    // 注册信号处理
    std::signal(SIGINT, handle_signal); // Ctrl+C
    std::signal(SIGTERM, handle_signal); // kill 命令

    tiny_http::ServerConfig config;

    // =====
    // 第1步：尝试加载配置文件
    // =====
    std::string config_path = "tiny_httpd.conf";

    for (int i = 1; i < argc; ++i) {
        if (std::string(argv[i]) == "-c" && i + 1 < argc) {
            config_path = argv[i + 1];
            break;
        }
    }

    try {
        config = tiny_http::parse_config_file(config_path);
        std::cout << "[配置] 已加载配置文件: " << config_path << "\n";
    } catch (const std::exception& e) {
        std::cerr << "[配置] 警告: " << e.what() << "\n";
        std::cerr << "[配置] 将使用默认配置启动\n";
    }
```

```

}

// =====
// 第2步：命令行参数覆盖（优先级高于配置文件）
// =====
tiny_http::apply_command_line(config, argc, argv);

// =====
// 第3步：打印最终生效的配置
// =====
std::cout << "=====\n";
std::cout << "Tiny HTTP Server 启动配置:\n";
std::cout << "  port          = " << config.port << "\n";
std::cout << "  document_root = " << config.document_root << "\n";
std::cout << "  worker_threads = " << config.worker_threads << "\n";
std::cout << "  keep_alive     = " << config.keep_alive_seconds << "s\n";
std::cout << "  max_events     = " << config.max_events << "\n";
std::cout << "  log_path       = " << config.log_path << "\n";
std::cout << "  log_level      = " << config.log_level << "\n";
std::cout << "  max_body_size  = " << config.max_body_size << " bytes\n";
if (config.rate_limit > 0) {
    std::cout << "  rate_limit     = " << config.rate_limit << " req/s\n";
}
if (!config.virtual_hosts.empty()) {
    std::cout << "  virtual_hosts = "
              << config.virtual_hosts.size() << " 个\n";
}
if (!config.locations.empty()) {
    std::cout << "  locations     = "
              << config.locations.size() << " 个\n";
}
std::cout << "=====\n";

// =====
// 第4步：启动服务器
// =====
tiny_http::LinuxHttpServer server(config);
g_server = &server;

if (!server.start()) {
    std::cerr << "启动失败! \n";
    return 1;
}

std::cout << "服务器已正常退出\n";
return 0;
}

```

关键设计决策：

1. 配置文件加载失败**不致命**——打印警告后用默认配置继续。这样即使配置文件写错了，服务器也能启动。
2. 命令行参数**后**于配置文件生效（`apply_command_line` 在 `parse_config_file` 之后调用），所以命令行优先级更高。
3. `-c` 参数允许指定任意路径的配置文件，方便测试不同配置。

按 `Esc`，输入 `:wq` 回车，保存退出。

验证编译：

```
cmake --build build
```

期望输出：编译成功。如果报 `config_parser.h` 找不到，检查文件是否创建在正确的路径 `include/tiny_http/config_parser.h`。

步骤 8：更新 CMakeLists.txt

要做什么：在 CMake 构建配置中添加新的源文件和测试目标。

8.1 打开文件

```
vim CMakeLists.txt
```

你看到的当前内容（升级前）：

```
cmake_minimum_required(VERSION 3.16)

project(TinyHttpServer LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_library(tiny_http_core
    src/http_parser.cpp
    src/http_response.cpp
    src/mime_types.cpp
    src/linux_server.cpp
    src/thread_pool.cpp
    src/async_logger.cpp
)

target_include_directories(tiny_http_core PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_compile_options(tiny_http_core PRIVATE
    $<$<CXX_COMPILER_ID:GNU,Clang>:-Wall -Wextra -Wpedantic>
)

add_executable(parser_tests
    tests/parser_tests.cpp
)
```

```

add_executable(tiny_httpd
    src/main.cpp
)

target_link_libraries(tiny_httpd PRIVATE
    tiny_http_core
)

target_link_libraries(parser_tests PRIVATE
    tiny_http_core
)

enable_testing()

add_test(NAME parser_tests COMMAND parser_tests)

```

8.2 需要做的修改

修改1: 在 `add_library(tiny_http_core)` 的源文件列表末尾添加一行:

```
src/config_parser.cpp
```

按 `i` 进入编辑模式, 在 `src/async_logger.cpp` 下面加一行 `src/config_parser.cpp`。

修改2: 在 `add_executable(parser_tests ...)` 之后, 添加新的测试可执行文件:

```

add_executable(config_parser_tests
    tests/config_parser_tests.cpp
)

```

修改3: 在 `target_link_libraries(parser_tests ...)` 之后, 添加链接配置:

```

target_link_libraries(config_parser_tests PRIVATE
    tiny_http_core
)

```

修改4: 在 `add_test(NAME parser_tests ...)` 之后添加:

```
add_test(NAME config_parser_tests COMMAND config_parser_tests)
```

8.3 最终 CMakeLists.txt 全貌

修改完成后应该像这样:

```

cmake_minimum_required(VERSION 3.16)

project(TinyHttpServer LANGUAGES CXX)

```

```

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_library(tiny_http_core
    src/http_parser.cpp
    src/http_response.cpp
    src/mime_types.cpp
    src/linux_server.cpp
    src/thread_pool.cpp
    src/async_logger.cpp
    src/config_parser.cpp
)

target_include_directories(tiny_http_core PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_compile_options(tiny_http_core PRIVATE
    $<$<CXX_COMPILER_ID:GNU,Clang>:-Wall -Wextra -Wpedantic>
)

add_executable(parser_tests
    tests/parser_tests.cpp
)

add_executable(config_parser_tests
    tests/config_parser_tests.cpp
)

add_executable(tiny_httpd
    src/main.cpp
)

target_link_libraries(tiny_httpd PRIVATE
    tiny_http_core
)

target_link_libraries(parser_tests PRIVATE
    tiny_http_core
)

target_link_libraries(config_parser_tests PRIVATE
    tiny_http_core
)

enable_testing()

add_test(NAME parser_tests COMMAND parser_tests)
add_test(NAME config_parser_tests COMMAND config_parser_tests)

```

按 `Esc`，输入 `:wq` 回车，保存退出。

验证编译：

```
cmake -S . -B build
cmake --build build
```

期望输出：CMake 配置和编译都成功。如果报 `tests/config_parser_tests.cpp` 找不到，很正常——我们下一步才创建它。

步骤 9：创建示例配置文件

要做什么：创建项目根目录下的 `tiny_httpd.conf` 示例文件。

```
vim tiny_httpd.conf
```

按 `i` 进入编辑模式，输入：

```
# =====
# Tiny HTTP Server 配置文件示例
# =====

server {
    # ---- 基础配置 ----
    listen 8080;
    root "www";
    worker_threads 4;
    keep_alive 15;
    max_events 1024;
    log_path "tiny_httpd.log";
    log_level info;

    # ---- 安全限制 ----
    max_body_size 1048576;          # 1MB
    # rate_limit 0;                  # 0 = 不限制（默认）

    # ---- 虚拟主机示例（取消注释以启用） ----
    # server_name example.com {
    #     root "/var/www/example";
    # }

    # ---- Location 规则示例（取消注释以启用） ----
    # location /api/ {
    #     proxy_pass "http://localhost:5000";
    # }
    # location /static/ {
    #     root "/srv/static";
    #     expires 7d;
    # }
}
```

按 `Esc`，输入 `:wq` 回车。

步骤 10：编写测试并验证

要做什么：创建配置解析器的单元测试，覆盖正常解析、错误处理、命令行覆盖等场景。

10.1 创建测试文件

```
touch tests/config_parser_tests.cpp
vim tests/config_parser_tests.cpp
```

按 **i** 进入编辑模式，输入以下完整测试代码：

```
#include "tiny_http/config_parser.h"

#include <cassert>
#include <cstdlib>
#include <ctime>
#include <fstream>
#include <iostream>
#include <sstream>

// 辅助函数：将字符串写入临时文件，返回文件路径
static std::string write_temp_config(const std::string& content) {
    std::string path = "/tmp/tiny_httpd_test_" +
        std::to_string(std::rand()) + ".conf";
    std::ofstream f(path);
    f << content;
    f.close();
    return path;
}

// 辅助函数：从字符串直接解析配置（不用手动创建文件）
static tiny_http::ServerConfig parse_string(const std::string& content) {
    std::string path = write_temp_config(content);
    auto config = tiny_http::parse_config_file(path);
    std::remove(path.c_str()); // 清理临时文件
    return config;
}

int main() {
    std::srand(static_cast<unsigned>(std::time(nullptr)));

    int passed = 0;
    int failed = 0;

    // 辅助 lambda：检查条件并打印结果
    auto check = [&](bool condition, const std::string& name) {
        if (condition) {
            std::cout << " [PASS] " << name << "\n";
            passed++;
        } else {
            std::cerr << " [FAIL] " << name << "\n";
            failed++;
        }
    };
}
```

```

    }
};

// =====
// 测试组1: 基本指令解析 (9个指令全部测试)
// =====
std::cout << "--- 测试组1: 基本指令 ---\n";

{
    auto config = parse_string(
        "server {\n"
        "    listen 9090;\n"
        "    root \"/tmp/test_www\";\n"
        "    worker_threads 8;\n"
        "    keep_alive 30;\n"
        "    max_events 2048;\n"
        "    log_path \"/var/log/tiny.log\";\n"
        "    log_level debug;\n"
        "    max_body_size 2097152;\n"
        "    rate_limit 100;\n"
        "}\n"
    );

    check(config.port == 9090, "port = 9090");
    check(config.document_root == "/tmp/test_www", "root = /tmp/test_www");
    check(config.worker_threads == 8, "worker_threads = 8");
    check(config.keep_alive_seconds == 30, "keep_alive = 30");
    check(config.max_events == 2048, "max_events = 2048");
    check(config.log_path == "/var/log/tiny.log", "log_path");
    check(config.log_level == "debug", "log_level = debug");
    check(config.max_body_size == 2097152, "max_body_size = 2MB");
    check(config.rate_limit == 100, "rate_limit = 100");
}

// =====
// 测试组2: location 块 (测试嵌套解析)
// =====
std::cout << "--- 测试组2: location 块 ---\n";

{
    auto config = parse_string(
        "server {\n"
        "    listen 8080;\n"
        "    location /api/ {\n"
        "        proxy_pass \"http://localhost:5000\";\n"
        "    }\n"
        "    location /static/ {\n"
        "        root \"/srv/static\";\n"
        "        expires 7d;\n"
        "    }\n"
        "}\n"
    );

    check(config.locations.size() == 2, "2 个 location");

    if (config.locations.size() >= 2) {
        check(config.locations[0].path == "/api/",
            "location[0] path = /api/");
    }
}

```

```

        check(config.locations[0].proxy_pass == "http://localhost:5000",
              "location[0] proxy_pass");
        check(config.locations[1].path == "/static/",
              "location[1] path = /static/");
        check(config.locations[1].root == "/srv/static",
              "location[1] root");
        check(config.locations[1].expires == "7d",
              "location[1] expires = 7d");
    }
}

// =====
// 测试组3: 虚拟主机 (server_name)
// =====
std::cout << "--- 测试组3: 虚拟主机 ---\n";

{
    auto config = parse_string(
        "server {\n"
        "    listen 8080;\n"
        "    server_name example.com {\n"
        "        root \"/var/www/example\";\n"
        "    }\n"
        "    server_name \"blog.example.com\" {\n"
        "        root \"/var/www/blog\";\n"
        "    }\n"
        "}\n"
    );

    check(config.virtual_hosts.size() == 2, "2 个虚拟主机");

    if (config.virtual_hosts.size() >= 2) {
        check(config.virtual_hosts[0].server_name == "example.com",
              "vh[0] name = example.com");
        check(config.virtual_hosts[0].root == "/var/www/example",
              "vh[0] root");
        check(config.virtual_hosts[1].server_name == "blog.example.com",
              "vh[1] name = blog.example.com");
    }
}

// =====
// 测试组4: 注释处理 (单行注释 + 行尾注释)
// =====
std::cout << "--- 测试组4: 注释 ---\n";

{
    auto config = parse_string(
        "# 这是注释行\n"
        "server {\n"
        "    # 端口号\n"
        "    listen 1234;    # 行尾注释\n"
        "    root \"/www\";    # 文档根目录\n"
        "}\n"
    );

    check(config.port == 1234, "注释行被正确跳过");
    check(config.document_root == "/www", "注释不影响指令解析");
}

```

```

}

// =====
// 测试组5: 空配置 (应返回默认值)
// =====
std::cout << "--- 测试组5: 空配置 ---\n";

{
    auto config = parse_string("");

    check(config.port == 8080, "空配置 port 保持默认");
    check(config.document_root == "www", "空配置 root 保持默认");
    check(config.worker_threads == 4, "空配置 workers 保持默认");
    check(config.locations.empty(), "空配置无 location");
    check(config.virtual_hosts.empty(), "空配置无虚拟主机");
}

// =====
// 测试组6: 错误处理 (未知指令 + 缺少分号)
// =====
std::cout << "--- 测试组6: 错误处理 ---\n";

{
    bool caught = false;
    try {
        parse_string("server { unknown_directive 123; }\n");
    } catch (const std::runtime_error& e) {
        caught = true;
        std::cout << "    预期错误: " << e.what() << "\n";
    }
    check(caught, "未知指令应抛出异常");

    caught = false;
    try {
        parse_string("server { listen 8080 }\n"); // 缺少分号
    } catch (const std::runtime_error& e) {
        caught = true;
        std::cout << "    预期错误: " << e.what() << "\n";
    }
    check(caught, "缺少分号应抛出异常");
}

// =====
// 测试组7: 命令行参数覆盖
// =====
std::cout << "--- 测试组7: 命令行覆盖 ---\n";

{
    tiny_http::ServerConfig config;
    config.port = 8080;
    config.document_root = "www";
    config.worker_threads = 4;

    // 模拟: ./tiny_httpd 9090 /var/www 8
    const char* argv[] = {"tiny_httpd", "9090", "/var/www", "8"};
    tiny_http::apply_command_line(
        config, 4, const_cast<char**>(argv));
}

```

```

        check(config.port == 9090, "命令行覆盖 port");
        check(config.document_root == "/var/www", "命令行覆盖 root");
        check(config.worker_threads == 8, "命令行覆盖 workers");
    }

    // =====
    // 测试组8: 配置文件不存在时抛异常
    // =====
    std::cout << "--- 测试组8: 配置文件不存在 ---\n";

    {
        bool caught = false;
        try {
            tiny_http::parse_config_file("/tmp/nonexistent_xyz.conf");
        } catch (const std::runtime_error& e) {
            caught = true;
            std::cout << "    预期错误: " << e.what() << "\n";
        }
        check(caught, "不存在的配置文件应抛出异常");
    }

    // =====
    // 结果汇总
    // =====
    std::cout << "\n===== \n";
    std::cout << "配置解析器测试结果: "
                << passed << " 通过, " << failed << " 失败\n";
    std::cout << "===== \n";

    return failed > 0 ? 1 : 0;
}

```

逐段解释:

- `write_temp_config()`: 把字符串写入 `/tmp` 下的随机文件名, 返回路径。用 `std::rand()` 避免多次调用冲突。
- `parse_string()`: 组合了"写入临时文件 → 解析 → 删除临时文件"三步。让测试代码可以直接写配置字符串, 不用手动创建文件。
- `check()` lambda: 统一测试断言格式。 `[&]` 捕获外部变量 (`passed` 和 `failed`) 的引用。
- 测试组1-5 验证正常解析, 测试组6 验证错误处理 (确认解析器不会默默忽略错误), 测试组7 验证命令行覆盖函数, 测试组8 验证文件不存在时的行为。

按 `Esc`, 输入 `:wq` 回车, 保存退出。

步骤 11: 编译并运行全部测试

```

# 重新配置 CMake (因为添加了新测试目标)
cmake -S . -B build
cmake --build build

```

```
# 运行新增的配置解析器测试
./build/config_parser_tests
```

期望输出：

```
--- 测试组1: 基本指令 ---
[PASS] port = 9090
[PASS] root = /tmp/test_www
[PASS] worker_threads = 8
[PASS] keep_alive = 30
[PASS] max_events = 2048
[PASS] log_path
[PASS] log_level = debug
[PASS] max_body_size = 2MB
[PASS] rate_limit = 100
--- 测试组2: location 块 ---
[PASS] 2 个 location
[PASS] location[0] path = /api/
[PASS] location[0] proxy_pass
[PASS] location[1] path = /static/
[PASS] location[1] root
[PASS] location[1] expires = 7d
--- 测试组3: 虚拟主机 ---
[PASS] 2 个虚拟主机
[PASS] vh[0] name = example.com
[PASS] vh[0] root
[PASS] vh[1] name = blog.example.com
--- 测试组4: 注释 ---
[PASS] 注释行被正确跳过
[PASS] 注释不影响指令解析
--- 测试组5: 空配置 ---
[PASS] 空配置 port 保持默认
[PASS] 空配置 root 保持默认
[PASS] 空配置 workers 保持默认
[PASS] 空配置无 location
[PASS] 空配置无虚拟主机
--- 测试组6: 错误处理 ---
    预期错误: 配置文件第 1 行: 未知的配置指令: unknown_directive
[PASS] 未知指令应抛出异常
    预期错误: 配置文件第 1 行: 期望 ';' , 但遇到了: }
[PASS] 缺少分号应抛出异常
--- 测试组7: 命令行覆盖 ---
[PASS] 命令行覆盖 port
[PASS] 命令行覆盖 root
[PASS] 命令行覆盖 workers
--- 测试组8: 配置文件不存在 ---
    预期错误: 无法打开配置文件: /tmp/nonexistent_xyz.conf
[PASS] 不存在的配置文件应抛出异常
```

```
=====
配置解析器测试结果: 25 通过, 0 失败
=====
```

如果看到任何 FAIL:

- 检查 `config_parser.cpp` 中 `parse_directive()` 的 if-else 是否覆盖了所有指令
- 检查 Lexer 的扫描函数是否正确处理了引号、数字、路径
- 检查 `expect()` 调用是否和语法规则匹配

运行原有测试确认没有破坏:

```
./build/parser_tests  
# 期望: parser tests passed (和升级前一模一样)
```

步骤 12: 端到端验证

测试1: 无配置文件启动 (应使用默认值):

```
# 临时改名让配置文件"不存在"  
mv tiny_httpd.conf tiny_httpd.conf.bak  
./build/tiny_httpd
```

期望输出: 先打印警告 [配置] 警告: 无法打开配置文件: `tiny_httpd.conf`, 然后打印默认配置 (port=8080, root=www, workers=4)。按 Ctrl+C 停止。

```
# 恢复配置文件  
mv tiny_httpd.conf.bak tiny_httpd.conf
```

测试2: 有配置文件启动:

```
./build/tiny_httpd
```

期望输出: [配置] 已加载配置文件: `tiny_httpd.conf`, 然后打印和配置文件一致的配置值。

测试3: 命令行覆盖配置文件:

```
./build/tiny_httpd 9090 /tmp/test_www 8
```

期望输出: port=9090 (不是配置文件的 8080), root=/tmp/test_www (不是配置文件的 www), workers=8 (不是配置文件的 4)。这证明命令行优先级高于配置文件。

测试4: 测试错误配置文件:

```
echo 'server { bad_directive 123; }' > /tmp/bad.conf  
./build/tiny_httpd -c /tmp/bad.conf
```

期望输出：带行号的错误信息 配置文件第 1 行：未知的配置指令：bad_directive，然后 将使用默认配置启动。

测试5：用 curl 确认服务器仍正常工作：

```
# 终端1: ./build/tiny_httpd
# 终端2:
curl -v http://localhost:8080/
# 期望: 返回 index.html, 和升级前一模一样
```

第 1 课练习答案

练习1：Lexer 的 skip_whitespace_and_comments() 中，为什么遇到 \n 要 line_++？如果不加会怎样？

答案：line_ 是用来在报错时告诉用户“第几行出了错”。如果不加 line_++，整个文件读完 line_ 始终是 1。

假设置配置文件有 50 行，第 50 行写错了指令。如果不维护行号，错误信息会显示 配置文件第 1 行：未知的配置指令：xxx —— 用户去第 1 行找，什么也没找到，一头雾水。正确的错误信息应该是 配置文件第 50 行：未知的配置指令：xxx，用户直接跳到第 50 行就能定位问题。

另外注意：skip_whitespace_and_comments() 中 # 注释的处理里没有 line_++。这是因为 # 跳到行尾就停了，换行符 \n 本身还在输入流里——下一轮循环遇到 \n 时自然会 line_++。如果注释处理里也加了 line_++，就会多算一行，行号又错了。

练习2：解析器为什么选择“配置文件加载失败不致命”的策略？工业级服务器 Nginx 的 nginx -t 是怎么做的？

答案：

我们的策略是“容错启动”——配置文件错了，打警告，用默认配置继续。这样服务器至少能跑起来，不会因为一个拼写错误就全站崩溃。开发阶段特别有用：你改了配置文件，不小心少写了一个分号，服务器不会挂掉，只是用回默认配置，你看警告就知道要修。

Nginx 的做法更严格——它提供了 nginx -t (test) 命令：

```
nginx -t
# 输出: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
#      nginx: configuration file /etc/nginx/nginx.conf test is successful
```

生产环境中，运维应该先 `nginx -t` 验证配置，确认无误后再 `nginx -s reload` 热加载。如果配置有语法错误，`nginx -t` 会明确报错，`reload` 也会被拒绝——这样就不会因为错误配置导致服务中断。

我们的项目也可以加一个类似的机制：检查 `-t` 命令行参数，只验证配置不启动服务器。但作为学习项目，当前的容错策略已经够用。

练习3：如果要支持 `include` 指令（引入另一个配置文件），Lexer 和 Parser 需要怎么改？

答案：

Nginx 支持 `include /etc/nginx/conf.d/*.conf;` 来引入其他配置文件。如果要实现这个功能，需要改动两个地方：

Lexer 层：需要支持“输入源栈”。当遇到 `include "other.conf"` 时，打开那个文件，把内容插入到当前位置。最简单的做法是在 Lexer 内部维护一个 `std::vector<std::pair<std::string, size_t>>` 栈——每个元素是（文件内容，当前读取位置）。当 `include` 触发时，把当前状态 push 到栈上，切换到新文件；新文件读完后 pop 回到原来的位置继续。

Parser 层：在 `parse_directive()` 中新增一个 `include` 指令的处理：

```
} else if (name == "include") {
    std::string include_path = current_.value;
    if (current_.type == TokenType::String) {
        expect(TokenType::String);
    } else {
        expect(TokenType::Keyword);
    }
    expect(TokenType::Semicolon);
    // 把 include_path 传给 Lexer, 让它加载新文件
    lexer_.push_source(include_path);
}
```

如果要支持通配符（`conf.d/*.conf`），还需要用 `glob()` 函数展开通配符，然后按字母顺序逐个加载。

第 1 课面试题点（含答案）

问题1: "为什么手写解析器而不用 JSON/YAML 库？"

答案：有三个理由。第一，这是学习项目，手写解析器本身就是非常好的练习——它让你理解“编程语言的编译器前端是怎么工作的”。Lexer（词法分析）和 Parser（语法分析）是编译原理的核心概念。你亲自实现一遍，以后遇到任何“解析文本格式”的需求都有思路。第二，避免了外部依赖——如果用 JSON 库需要引入 `nlohmann/json`，用 YAML 需要 `yaml-cpp`。我们的项目目标是“从零构

建”，能自己写的就不依赖第三方。第三，我们的配置格式是专门为 HTTP 服务器设计的，比 JSON 更简洁（不需要大量引号和大括号），比 YAML 更可控（缩进敏感容易出错）。

问题2: "递归下降解析器的优缺点？"

答案：优点：直观——每个语法结构对应一个函数，代码结构一目了然；易于调试——可以在任意解析函数里加断点或日志；错误信息友好——可以精确到行号和期望的 `token` 类型（我们的 `expect()` 函数做到了这一点）；手写代码量可控——对于简单文法不需要引入 `parser generator`。

缺点：手写代码量大——相比用 `yacc/bison` 这种工具自动生成，手写确实更费时；对左递归文法需要特殊处理——比如如果定义 `expr = expr + term | term`，递归下降会无限循环（因为进入 `parse_expr()` 后又立刻调用自己），需要手动改写为右递归或用循环；嵌套层级深时函数调用栈也很深——但配置文件的嵌套通常不超过 3 层，完全不是问题。

对于我们的配置语言这种简单文法（不超过 10 种指令，嵌套不超过 3 层），递归下降是最佳选择——出错信息友好、代码好维护。

问题3: "配置文件加载失败为什么还能启动？"

答案：这是一种容错设计。核心原则是"不要让配置错误导致服务完全不可用"。

实际场景：假设你的服务器正在运行，你修改了配置文件准备重启。不小心少写了一个分号。如果解析失败直接退出，服务器就挂了——用户访问不到、监控报警、老板打电话。但如果采用容错策略，服务器打印一条警告、用默认配置或上次的配置继续运行，至少服务没有中断。

生产环境中的最佳实践是 Nginx 的 `nginx -t && nginx -s reload` 模式——先验证、再热加载。我们的项目后续也可以加 `-t` 参数来实现同样的安全校验。但在没有热加载能力的当前阶段，"配置文件错了但服务器还能用默认配置跑"比"配置文件错了服务器直接崩溃"要好得多。

第 2 课：压力测试基线

本节目标

用标准压测工具建立服务器的性能基线数据，学会用分析工具定位瓶颈。**这一课不写代码**——你只需要操作终端，运行命令，记录数据。这些数据在后面的第 5 课（sendfile）会作为"优化前 vs 优化后"的对比基准。

理论知识：压测的核心指标

指标	含义	为什么重要
QPS (Queries Per Second)	每秒处理多少请求	衡量吞吐量，越高越好
Latency (延迟)	从发请求到收到响应的耗时	用户体验，越低越好
P50	50% 请求的延迟不超过这个值	大多数用户的体验
P99	99% 请求的延迟不超过这个值	极端情况，反映系统稳定性
Transfer/sec	每秒传输多少数据	衡量带宽利用率
Socket errors	连接失败数	高并发下是否稳定

压测工具对比：

工具	特点	推荐场景
wrk	轻量、高性能、支持 Lua 脚本	日常压测首选，我们用它
wrk2	控制恒定吞吐量	分析延迟分布
ab (ApacheBench)	经典、简单	快速验证，备用
hey	Go 实现，支持 HTTP/2	HTTP/2 压测

我们用 **wrk**，因为它最轻量、输出信息最实用。

涉及文件

操作	文件	说明
新建	www/1b.txt	1 字节小文件
新建	www/1k.bin	1KB 中等文件
新建	www/1m.bin	1MB 大文件
新建	www/test.html	模拟真实网页
新建	benchmarks/run_benchmark.sh	自动化压测脚本
新建	benchmarks/baseline_result.txt	基线数据

步骤 1：安装压测工具

打开终端，执行：

```
# 安装 wrk
sudo apt install -y wrk

# 如果 apt 源里没有 wrk（某些发行版），从源码编译：
# git clone https://github.com/wg/wrk.git
# cd wrk && make && sudo cp wrk /usr/local/bin/

# 安装 ab 作为备用
sudo apt install -y apache2-utils

# 确认安装成功
wrk --version
# 期望输出: wrk 4.x.x ...

ab -V
# 期望输出: This is ApacheBench, Version 2.3 ...
```

步骤 2：准备测试文件

```
cd ~/projects/tiny-http-server

# 小文件（1 字节）——测试极限 QPS
echo -n "x" > www/1b.txt

# 中等文件（1KB）——测试典型静态资源
dd if=/dev/urandom of=www/1k.bin bs=1024 count=1 2>/dev/null
```

```
# 大文件（1MB）——测试吞吐量上限
dd if=/dev/urandom of=www/1m.bin bs=1048576 count=1 2>/dev/null

# HTML 文件——模拟真实网页请求
cat > www/test.html << 'EOF'
<!DOCTYPE html>
<html><head><title>Test</title></head>
<body><h1>Test Page</h1><p>Hello from tiny_httpd</p></body>
</html>
EOF

# 确认文件创建成功
ls -lh www/1b.txt www/1k.bin www/1m.bin www/test.html
# 期望：看到 4 个文件，大小分别是 1B、1KB、1MB、约100B
```

步骤 3：启动服务器并运行基线压测

你需要两个终端窗口。

终端1（启动服务器）：

```
cd ~/projects/tiny-http-server
./build/tiny_httpd
# 看到 "服务器已正常退出" 之前不要关闭
```

终端2（运行压测）：

```
cd ~/projects/tiny-http-server
mkdir -p benchmarks

# 将全部 6 项测试结果保存到基线文件
{
    echo "====="
    echo "Tiny HTTP Server 性能基线测试"
    echo "日期: $(date)"
    echo "版本: 基础版（第1课完成后，优化前）"
    echo "====="
    echo ""

    echo "### 测试1: 小文件短连接（最严格条件）###"
    wrk -t4 -c100 -d30s http://localhost:8080/1b.txt
    echo ""

    echo "### 测试2: 小文件长连接（模拟浏览器）###"
    wrk -t4 -c100 -d30s -H "Connection: keep-alive" http://localhost:8080/1b.txt
    echo ""

    echo "### 测试3: 1KB文件 ###"
    wrk -t4 -c100 -d30s http://localhost:8080/1k.bin
    echo ""
```

```

echo "### 测试4: 1MB大文件 (测试吞吐量) ###"
wrk -t4 -c10 -d30s http://localhost:8080/lm.bin
echo ""

echo "### 测试5: HTML页面 (真实场景) ###"
wrk -t4 -c100 -d30s http://localhost:8080/test.html
echo ""

echo "### 测试6: 404页面 (错误路径) ###"
wrk -t4 -c100 -d30s http://localhost:8080/notfound.html
echo ""

} > benchmarks/baseline_result.txt

# 查看结果
cat benchmarks/baseline_result.txt

```

参数解释：

- `-t4`：4 个压测线程。一般设为核心数（`nproc` 查看）。
- `-c100`：100 个并发 TCP 连接。模拟 100 个用户同时访问。
- `-d30s`：持续 30 秒。时间越长数据越稳定，排除偶然波动。
- `-H "Connection: keep-alive"`：添加 HTTP 头。不加的话 wrk 默认也是 keep-alive，但显式声明更清晰。
- 大文件测试用 `-c10` 而不是 `-c100`，因为 100 个连接同时下载 1MB 文件会把带宽打满，QPS 数据没有意义。大文件主要看 **Transfer/sec**（吞吐量）。

理解 wrk 输出：

```

Running 30s test @ http://localhost:8080/lb.txt
 4 threads and 100 connections
  Thread Stats   Avg      Stdev     Max    +/-  Stdev
   Latency    1.23ms    2.45ms   45.67ms   89.12%
   Req/Sec    23.45k    3.21k   34.56k   72.34%
2800000 requests in 30.00s, 234.00MB read
Requests/sec: 93333.33
Transfer/sec:      7.80MB

```

关键数字：

- **Requests/sec** (QPS)：这个服务器每秒能处理约 9.3 万个请求
- **Latency Avg**：平均延迟 1.23 毫秒
- **Latency Max**：最大延迟 45.67 毫秒（P99 约在这个范围）
- **Transfer/sec**：每秒传输 7.8MB 数据

记下你的实际数字！ 特别是"小文件长连接"的 Requests/sec——这是后续第 5 课 sendfile 优化后的对比基准。

步骤 4：系统调用分析

用 strace 看服务器把 CPU 时间花在哪些系统调用上：

```
# 终端1: 启动服务器
./build/tiny_httpd &
SERVER_PID=$!

# 终端2: 先启动压测
wrk -t4 -c100 -d60s http://localhost:8080/1b.txt &

# 终端3 (或等几秒后在终端2按 Ctrl+C 再执行)：统计系统调用
sudo strace -c -p $SERVER_PID
# 等待约 15 秒，按 Ctrl+C 停止

# 清理
kill $SERVER_PID
```

你会看到类似这样的输出：

% time	seconds	usecs/call	calls	errors	syscall
35.23	0.123456	12	10234		recvfrom
28.12	0.098765	9	10987		sendto
15.34	0.053876	13	4123		epoll_wait
10.21	0.035851	8	4482		read
5.67	0.019912	5	3984		write
2.11	0.007410	3	2470		accept4

解读：

- `recvfrom` 和 `sendto` 占用了最多时间——这是网络 I/O，合理
- `epoll_wait` 占比 15%——事件等待的开销
- `read` 和 `write` 是文件 I/O——这就是后面 `sendfile` 要优化的目标（把 `read+write` 合并成 `sendfile`）

把这些数据记录下来——后面第 5 课做完 `sendfile` 后再跑一次 `strace`，你会看到 `read/write` 的占比大幅下降。

步骤 5：创建自动化压测脚本

创建一个脚本，以后每次优化后跑一遍，自动保存结果：

```
vim benchmarks/run_benchmark.sh
```

按 `i` 进入编辑模式，输入：

```
#!/bin/bash
# 自动化压测脚本 — 每次优化后运行，对比性能变化
set -e

PORT=${1:-8080}
RESULT_DIR="benchmarks/results_$(date +%Y%m%d_%H%M%S)"
mkdir -p "$RESULT_DIR"

echo "=== Tiny HTTP Server 压测 ==="
echo "目标端口: $PORT"
echo "结果目录: $RESULT_DIR"
echo ""

# 确认服务器在运行
if ! curl -s -o /dev/null "http://localhost:$PORT/"; then
    echo "错误: 服务器未运行在端口 $PORT"
    exit 1
fi

echo "--- 测试1: 小文件短连接 ---"
wrk -t4 -c100 -d30s "http://localhost:$PORT/1b.txt" | tee "$RESULT_DIR/1_小文件短连接.txt"

echo ""
echo "--- 测试2: 小文件长连接 ---"
wrk -t4 -c100 -d30s -H "Connection: keep-alive" "http://localhost:$PORT/1b.txt" | tee "$RESULT_DIR/2_小文件长连接.txt"

echo ""
echo "--- 测试3: 1KB文件 ---"
wrk -t4 -c100 -d30s "http://localhost:$PORT/1k.bin" | tee "$RESULT_DIR/3_1k文件.txt"

echo ""
echo "--- 测试4: 1MB大文件 ---"
wrk -t4 -c100 -d30s "http://localhost:$PORT/1m.bin" | tee "$RESULT_DIR/4_1m文件.txt"

echo ""
echo "--- 测试5: HTML页面 ---"
wrk -t4 -c100 -d30s "http://localhost:$PORT/test.html" | tee "$RESULT_DIR/5_html页面.txt"

echo ""
echo "--- 测试6: 404页面 ---"
wrk -t4 -c100 -d30s "http://localhost:$PORT/notfound.html" | tee "$RESULT_DIR/6_404页面.txt"

echo ""
echo "=== 全部测试完成 ==="
echo "结果保存在: $RESULT_DIR"
echo ""
echo "快速对比 (Requests/sec) : "
grep -h "Requests/sec" "$RESULT_DIR/*.txt"
```

按 `Esc`，输入 `:wq` 回车。

```
chmod +x benchmarks/run_benchmark.sh
```

以后每次优化后，只需：

```
# 终端1: 启动新版本服务器
./build/tiny_httpd

# 终端2: 一键压测
./benchmarks/run_benchmark.sh
```

每次运行都会创建带时间戳的结果目录，形成性能演进的时间线。

验证清单

完成以下所有项，第 2 课才算完成：

- [] `wrk` 安装成功，`wrk --version` 有输出
- [] 6 项测试全部运行，数据已保存到 `benchmarks/baseline_result.txt`
- [] 你知道当前"小文件长连接"的 QPS 大约是多少
- [] 你知道当前"1MB大文件"的吞吐量大约是多少 MB/s
- [] `strace -c` 的结果已记录，知道 CPU 时间主要花在哪些系统调用上
- [] `benchmarks/run_benchmark.sh` 脚本可执行且能正常运行

第 2 课练习答案

练习1：用 `wrk -t4 -c100` 和 `wrk -t2 -c500` 分别压测，哪个 QPS 更高？为什么？

答案：通常 `-t4 -c100` 的 QPS 更高。

原因分析：假设你的 CPU 是 4 核的。`-t4` 让 4 个核心都参与压测，每个核心处理约 25 个连接（ $100 \div 4$ ），负载均衡。`-t2` 只用 2 个核心，每个核心要处理 250 个连接（ $500 \div 2$ ），线程上下文切换的开销急剧增加。虽然总连接数更多（ $500 > 100$ ），但线程数太少成了瓶颈。

你可以实际测试验证：先跑 `wrk -t4 -c100 -d10s`，再跑 `wrk -t2 -c500 -d10s`，对比 Requests/sec。在大多数系统上前者会更高。

练习2：对比长连接和短连接的 QPS 差异，解释差异原因。

答案：长连接的 QPS 远高于短连接（通常 2-5 倍）。

原因在于 **TCP 握手的开销**。短连接模式下，每个 HTTP 请求需要：

1. TCP 三次握手 (SYN → SYN-ACK → ACK)
2. 发送 HTTP 请求 + 接收响应
3. TCP 四次挥手 (FIN → ACK → FIN → ACK)

其中步骤 1 和步骤 3 是"纯开销"——它们不传输任何 HTTP 数据。对于 1 字节的小文件来说，TCP 握手和挥手的开销甚至比传输 HTTP 数据本身还大。

长连接模式下，100 个连接各做一次 TCP 握手，然后在 30 秒内复用这些连接发送大量请求——握手开销被摊销了。

练习3：在压测期间用 `top` 观察 `tiny_httpd` 的 CPU 占用率，是 100% 还是远低于？为什么？

答案：通常远低于 100%（可能在 20%-60% 之间）。

原因：服务器是 **I/O 密集型**而不是 CPU 密集型。大量 CPU 时间花在等待上——等待 `epoll_wait()` 返回事件、等待 `recv()` 从内核缓冲区读数据、等待 `send()` 把数据写入内核缓冲区。这些 I/O 操作期间，CPU 是空闲的（或者被操作系统调度给其他进程）。

这也是性能优化的方向：减少 I/O 等待时间（比如用 `sendfile` 减少数据拷贝）、减少系统调用次数（比如用更大的 buffer 批量读写）。`strace -c` 的输出印证了这一点——`recvfrom`、`sendto`、`epoll_wait` 占了大部分时间，这些都是 I/O 系统调用。

第 2 课面试要点（含答案）

问题1："你怎么压测的？用的什么工具？"

答案：我用 `wrk` 做压测。测试方案是：准备 1B、1KB、1MB 三种大小的静态文件，分别测试短连接和长连接两种模式。每个测试跑 30 秒，用 4 线程 100 并发（小文件）或 10 并发（大文件），记录 QPS、平均延迟、P99 延迟、吞吐量。

我建了一个自动化脚本 `run_benchmark.sh`，每次优化后跑一遍，结果按时间戳存档，形成完整的性能演进数据链。优化前后有具体的数字对比——比如做完 `sendfile` 后，1MB 文件的吞吐量从 X MB/s 提升到 Y MB/s，提升了 Z 倍。

问题2："你服务器的瓶颈在哪？怎么分析出来的？"

答案：通过 `strace` 做系统调用分析。`strace -c -p <pid>` 统计每种系统调用的耗时占比，发现 `read` 和 `write` 系统调用占了很大比例——这说明文件 I/O 是一个瓶颈。另外 `epoll_wait` 也有一定占比。后续通过 `sendfile` 零拷贝优化了文件传输路径，`read+write` 变成了一次 `sendfile`，吞吐量提升了约 2-4 倍。

如果要进一步分析 CPU 热点，可以用 `perf record -g` 做 CPU 采样然后生成火焰图，定位到具体是哪个函数占用了最多 CPU 时间。

第3课：Chunked 传输编码

本节目标

实现 HTTP/1.1 分块传输编码的两个方向：

1. **解析方向**：客户端发来的 chunked 请求体 → 还原成完整数据
2. **生成方向**：服务器流式返回数据 → 编码为 chunked 格式

不支持 chunked 就不能说完整支持 HTTP/1.1。这也是后续实现反向代理（第 13 课）和 FastCGI（第 12 课）的基础——那些场景经常需要流式传输。

理论知识：为什么需要分块传输

问题：服务器要返回动态内容（比如数据库查询结果），但生成开始时不知道总大小。

Content-Length 方式要求你**先知道总大小**才能发响应头，这对动态内容是不可能的。Chunked 方式允许你**边生成边发送**——先生成一块就发一块，全部发完后发一个大小为 0 的终止块。

Chunked 编码格式：

```
HTTP/1.1 200 OK
Transfer-Encoding: chunked      ← 告诉客户端"我用分块传输"
Content-Type: text/plain

5\r\n                          ← 第1块大小：5字节（十六进制）
hello\r\n                      ← 第1块数据
A\r\n                          ← 第2块大小：10字节（A=10）
1234567890\r\n                ← 第2块数据
0\r\n                          ← 终止块：大小为0
\r\n                          ← 终止块的尾随空行（trailer 为空）
```

关键规则：

- 每个块 = `<十六进制大小>\r\n<数据>\r\n`
- 最后一个块大小必须是 `0`，后面跟一个空行
- 块大小可以跟扩展参数：`A;foo=bar\r\n`（分号后的部分可选，要能跳过）
- 终止块后面可以有 trailer headers（极少使用，我们忽略）

解析状态机：

```
初始：期望读块大小行
|
└→ 找到 \r\n → 解析十六进制大小
```

```

└─ 大小>0 → 期望读"大小"字节数据 + \r\n → 追加到 body → 回到初始
└─ 大小=0 → 期望读尾部空行 → 完成!
└─ 数据不够 → 返回 Incomplete (等 recv 更多数据)

```

涉及文件

操作	文件	说明
修改	include/tiny_http/http_request.h	增加 is_chunked 字段
修改	src/http_parser.cpp	新增 chunked 解析函数 + 修改 parse()
修改	include/tiny_http/http_response.h	新增 serialize_headers() 方法
修改	src/http_response.cpp	实现 chunked 序列化逻辑
修改	tests/parser_tests.cpp	增加 chunked 测试用例

步骤 1: 扩展 HttpRequest 结构体

```
vim include/tiny_http/http_request.h
```

按 `i` 进入编辑模式。在 `struct HttpRequest` 的 `std::string body;` 下面加一行:

```
bool is_chunked {false};           // 是否是分块传输
```

改完后 `HttpRequest` 的字段部分像这样:

```

struct HttpRequest {
    std::string method;
    std::string target;
    std::string version;
    std::map<std::string, std::string> headers;
    std::string body;
    bool is_chunked {false};           // ← 新增

    bool keep_alive() const;
    std::string header(const std::string& key) const;
};

```

按 `Esc`，输入 `:wq` 回车。

步骤 2：在 http_parser.cpp 中实现 chunked 解析

```
vim src/http_parser.cpp
```

第1步：在文件顶部 `namespace {` 匿名命名空间中（`trim()` 和 `lower()` 函数的后面），新增 `parse_chunked_body()` 函数。

按 **G** 跳到文件末尾，在 `} // namespace` 之前按 **i** 插入：

```
// 解析 chunked body
// 返回值: {是否成功, 是否完成, 已解析的body, 已消费字节数, 错误信息}
struct ChunkParseResult {
    bool ok = false;
    bool complete = false;
    std::string body;
    size_t consumed = 0;
    std::string error;
};

ChunkParseResult parse_chunked_body(const std::string& buffer) {
    ChunkParseResult result;
    size_t pos = 0;
    std::string body;

    while (true) {
        // ---- 第1步: 读取块大小行 ----
        size_t line_end = buffer.find("\r\n", pos);
        if (line_end == std::string::npos) {
            // 块大小行还不完整, 等更多数据
            result.ok = true;
            result.complete = false;
            result.body = body;
            result.consumed = pos;
            return result;
        }

        // 解析十六进制块大小
        std::string size_line = buffer.substr(pos, line_end - pos);

        // 去掉扩展参数 (分号之后的部分, 如 "A;foo=bar")
        size_t semi = size_line.find(';');
        if (semi != std::string::npos) {
            size_line = size_line.substr(0, semi);
        }

        size_t chunk_size = 0;
        try {
            chunk_size = std::stoul(size_line, nullptr, 16);
        } catch (...) {
            result.ok = false;
            result.error = "invalid chunk size: " + size_line;
            return result;
        }
    }
}
```

```

pos = line_end + 2; // 跳过块大小行的 \r\n

// ---- 第2步: 如果大小为0, 这是终止块 ----
if (chunk_size == 0) {
    // 读取尾部 (trailer headers + 空行)
    // 简化处理: 跳过所有 trailer 行直到空行
    while (pos + 1 < buffer.size()) {
        if (buffer[pos] == '\r' && buffer[pos+1] == '\n') {
            pos += 2;
            break; // 找到了终止空行
        }
        size_t next = buffer.find("\r\n", pos);
        if (next == std::string::npos) {
            // trailer 还不完整
            result.ok = true;
            result.complete = false;
            result.body = body;
            result.consumed = pos;
            return result;
        }
        pos = next + 2; // 跳过这行 trailer
    }

    result.ok = true;
    result.complete = true;
    result.body = body;
    result.consumed = pos;
    return result;
}

// ---- 第3步: 读取块数据 ----
if (buffer.size() - pos < chunk_size + 2) {
    // 块数据还不完整 (+2 是数据后的 \r\n)
    result.ok = true;
    result.complete = false;
    result.body = body;
    result.consumed = pos;
    return result;
}

// 追加块数据
body.append(buffer, pos, chunk_size);
pos += chunk_size;

// 跳过块数据后的 \r\n
if (buffer.substr(pos, 2) != "\r\n") {
    result.ok = false;
    result.error = "missing CRLF after chunk data";
    return result;
}
pos += 2;

// 继续循环, 读取下一个块
}
}

```

逐段解释：

- 外层 `while(true)`：循环读取一个又一个块，直到遇到大小为 0 的终止块。
- 第1步：用 `find("\r\n")` 找块大小行结尾。找到后用 `std::stoul(line, nullptr, 16)` 把十六进制字符串转成数字（"A" → 10）。分号之后的内容（chunk extension）直接丢弃——我们不需要解析它，只需要跳过。
- 第2步：`chunk_size == 0` 是终止块。后面可能有 trailer headers（键值对行），我们逐行跳过直到遇到空行（`\r\n` 单独一行）。大部分客户端不发送 trailer，所以这个逻辑通常只跳过最后的 `\r\n`。
- 第3步：确保缓冲区有足够数据（`chunk_size + 2` 字节），追加到 body，跳过数据后的 `\r\n`，继续循环。
- 数据不够就返回 `complete = false` ——这是和 HTTP 解析器一致的设计：让调用者继续 `recv` 更多数据，下次再调。

第2步：修改 `parse()` 函数，在 body 解析部分增加 chunked 判断。

在 `HttpParser::parse()` 中找到 body 解析的代码段（`size_t body_start = header_end + 4`；附近）。在 `body_start` 定义之后、原来的 `size_t content_length = 0`；之前，插入 chunked 判断：

```
size_t body_start = header_end + 4;

// ---- 判断是否 chunked 传输 ----
bool is_chunked = false;
std::string transfer_encoding = result.request.header("Transfer-Encoding");
if (!transfer_encoding.empty()) {
    // 转小写比较（HTTP 头字段名不区分大小写）
    std::transform(transfer_encoding.begin(), transfer_encoding.end(),
                  transfer_encoding.begin(),
                  [](unsigned char c) { return std::tolower(c); });
    if (transfer_encoding.find("chunked") != std::string::npos) {
        is_chunked = true;
    }
}

if (is_chunked) {
    // === Chunked 传输分支 ===
    std::string body_data = buffer.substr(body_start);
    auto chunk_result = parse_chunked_body(body_data);

    if (!chunk_result.ok) {
        result.status = ParseStatus::BadRequest;
        result.error = chunk_result.error;
        return result;
    }

    if (!chunk_result.complete) {
        result.status = ParseStatus::Incomplete;
        return result;
    }
}
```

```

        result.request.body = std::move(chunk_result.body);
        result.request.is_chunked = true;
        result.consumed = body_start + chunk_result.consumed;
        result.status = ParseStatus::Complete;
        return result;
    }

    // === Content-Length 分支（原逻辑，不动）===
    size_t content_length = 0;
    // ... 后面的代码不变 ...

```

这段代码的核心逻辑：先检查 `Transfer-Encoding` 头是否包含 `"chunked"`，如果是就走新的 `chunked` 分支，否则走原来的 `Content-Length` 分支。注意 `find("chunked")` 用 `std::string::find()` 而不是 `==`，因为 `Transfer-Encoding` 的值可能是 `"chunked, gzip"` 这种组合形式（虽然少见）。

按 `Esc`，输入 `:wq` 回车。

步骤 3：实现 Chunked 响应生成

```
vim include/tiny_http/http_response.h
```

按 `i` 进入编辑模式。在 `public:` 区域，`serialize()` 声明下面新增两个方法：

```

// 只序列化状态行和头部（不含 body）
// 在 sendfile 零拷贝场景中有用：先发 headers，再用 sendfile 发文件
std::string serialize_headers() const;

```

在 `private:` 区域新增字段：

```
bool is_chunked_ {false};
```

按 `Esc`，输入 `:wq` 回车。

现在修改实现文件：

```
vim src/http_response.cpp
```

按 `i` 进入编辑模式。新增 `serialize_headers()` 的实现，并修改 `serialize()` 支持 `chunked` 模式：

```

std::string HttpResponse::serialize_headers() const {
    std::ostringstream response;

```

```

        response << "HTTP/1.1 " << status_ << ' ' << reason_ << "\r\n";
        for (const auto& [key, value] : headers_) {
            response << key << ": " << value << "\r\n";
        }
        response << "Connection: "
            << (keep_alive_ ? "keep-alive" : "close") << "\r\n";
        response << "\r\n";
        return response.str();
    }

    std::string HttpResponse::serialize() const {
        if (!is_chunked_) {
            // 原逻辑: 普通模式
            std::ostringstream response;
            response << "HTTP/1.1 " << status_ << ' ' << reason_ << "\r\n";
            for (const auto& [key, value] : headers_) {
                response << key << ": " << value << "\r\n";
            }
            response << "Connection: "
                << (keep_alive_ ? "keep-alive" : "close") << "\r\n";
            response << "\r\n";
            response << body_;
            return response.str();
        }

        // Chunked 模式: headers + 各 chunk + 终止块
        std::ostringstream response;
        response << "HTTP/1.1 " << status_ << ' ' << reason_ << "\r\n";
        for (const auto& [key, value] : headers_) {
            response << key << ": " << value << "\r\n";
        }
        response << "Connection: "
            << (keep_alive_ ? "keep-alive" : "close") << "\r\n";
        response << "\r\n";

        // 逐个输出 chunk
        for (const std::string& chunk : chunks_) {
            response << std::hex << chunk.size() << "\r\n";
            response << chunk << "\r\n";
        }

        // 终止块
        response << "0\r\n\r\n";

        return response.str();
    }
}

```

还需要在 `set_body()` 中维护 `is_chunked_`。找到 `set_body()` 的实现，检查它的逻辑：当前 `set_body()` 直接设置 `body_` 并加 `Content-Length` 头。我们新增一个 `set_chunked()` 方法用于 `chunked` 模式。

在 `http_response.h` 中新增声明：

```

// 设置 chunked 传输模式（用于流式响应）
// 调用后应使用 add_chunk() 添加数据块，最后调用 serialize()

```

```
void set_chunked(bool chunked);
void add_chunk(std::string data);
```

在 `http_response.cpp` 中实现：

```
void HttpResponse::set_chunked(bool chunked) {
    is_chunked_ = chunked;
    if (chunked) {
        set_header("Transfer-Encoding", "chunked");
        headers_.erase("Content-Length"); // chunked 不应有 Content-Length
    }
}

void HttpResponse::add_chunk(std::string data) {
    chunks_.push_back(std::move(data));
}
```

还需要在头文件的 `private:` 区域新增 `chunks_` 成员：

```
std::vector<std::string> chunks_; // chunked 模式下的数据块列表
```

按 `Esc`，输入 `:wq` 回车。

步骤 4：添加测试用例

```
vim tests/parser_tests.cpp
```

按 `G` 跳到文件末尾，在 `main()` 的 `return 0;` 之前，按 `i` 新增以下测试：

```
// 测试5: Chunked 传输 — 基本解析
{
    const std::string raw =
        "POST /upload HTTP/1.1\r\n"
        "Transfer-Encoding: chunked\r\n"
        "\r\n"
        "5\r\n"
        "hello\r\n"
        "6\r\n"
        " world\r\n"
        "0\r\n"
        "\r\n";
    const ParseResult r = parser.parse(raw);
    assert(r.status == ParseStatus::Complete);
    assert(r.request.body == "hello world");
    std::cout << " [OK] chunked basic: " << r.request.body << "\n";
}

// 测试6: Chunked 传输 — 带扩展参数
```

```

{
    const std::string raw =
        "POST /upload HTTP/1.1\r\n"
        "Transfer-Encoding: chunked\r\n"
        "\r\n"
        "A;foo=bar\r\n"
        "0123456789\r\n"
        "0\r\n"
        "\r\n";
    const ParseResult r = parser.parse(raw);
    assert(r.status == ParseStatus::Complete);
    assert(r.request.body == "0123456789");
    std::cout << " [OK] chunked extension: " << r.request.body << "\n";
}

// 测试7: Chunked 传输 — 分片到达 (半包)
{
    const std::string part1 =
        "POST /upload HTTP/1.1\r\n"
        "Transfer-Encoding: chunked\r\n"
        "\r\n"
        "5\r\n"
        "hel"; // ← 块数据只到一半

    const ParseResult r1 = parser.parse(part1);
    assert(r1.status == ParseStatus::Incomplete);

    const std::string part2 = part1 + "lo\r\n0\r\n\r\n";
    const ParseResult r2 = parser.parse(part2);
    assert(r2.status == ParseStatus::Complete);
    assert(r2.request.body == "hello");
    std::cout << " [OK] chunked partial receive\n";
}

```

按 `Esc`，输入 `:wq` 回车。

步骤 5: 编译并验证

```

cmake --build build
./build/parser_tests

```

期望输出:

```

[OK] chunked basic: hello world
[OK] chunked extension: 0123456789
[OK] chunked partial receive
parser tests passed

```

用 `curl` 手动测试 `chunked` 请求:

```
# 终端1: 启动服务器
./build/tiny_httpd

# 终端2: 发送 chunked 编码的 POST 请求
echo -ne "5\r\nhello\r\n6\r\n world\r\n0\r\n\r\n" | \
  curl -v -X POST \
    -H "Transfer-Encoding: chunked" \
    --data-binary @- \
    http://localhost:8080/
```

第 3 课练习答案

练习1: 为什么 chunked 编码中块大小用十六进制而不用十进制?

答案: HTTP/1.1 规范 (RFC 7230 § 4.1) 就是这样定义的, 没有特别的深层原因。但从工程角度看, 十六进制有一些优势: 更紧凑 (FF 比 255 少一个字符)、解析简单 (不需要区分数字和其他字符——十六进制只用到 0-9a-f)、与 URI 编码等其他 HTTP 子系统的风格一致。不过最关键的原因还是“规范这么写的”——兼容性要求必须用十六进制。

练习2: 如果客户端同时发送了 Content-Length 和 Transfer-Encoding: chunked, 应该以哪个为准?

答案: 以 Transfer-Encoding: chunked 为准。RFC 7230 § 3.3.3 明确规定:

If a message is received with both a Transfer-Encoding and a Content-Length header field, the Transfer-Encoding overrides the Content-Length. Such a message might indicate an attempt to perform request smuggling.

Content-Length 必须被忽略。这是为了防止 **HTTP 请求走私攻击** (Request Smuggling) ——攻击者构造一个同时包含两个头的请求, 如果前端代理和后端服务器对“以哪个为准”的判断不一致, 攻击者就能在同一个连接中“夹带”额外的恶意请求。

练习3: parse_chunked_body() 在遇到 "A;foo=bar\r\n" 这样的扩展参数时的处理流程。

答案: 处理流程如下:

1. 找到行尾 \r\n, 提取 "A;foo=bar" 作为 size_line
2. 查找分号 ; 的位置, semi = 1 ('A' 后面的位置)
3. 截取分号前的部分: size_line = "A" (丢弃 ";foo=bar")
4. 用 std::stoul("A", nullptr, 16) 解析出 chunk_size = 10

5. 正常读取 10 字节数据 + `\r\n`

扩展参数被完全忽略——我们不解析它，也不检查它是否合法。这是正确的做法：扩展参数是可选的对等端元数据，代理和中间件应该透传但不应该尝试解析。

第 3 课面试要点（含答案）

问题1: "HTTP 解析器如何处理半包？具体说说 chunked 的情况。"

答案：解析器返回 `Incomplete` 状态，事件循环继续 `recv()` 追加数据到 `conn.in` 缓冲区，下次循环再调 `parse()` 重新解析。对于 chunked 编码，半包可能发生在三种情况：块大小行不完整（`find("\r\n")` 返回 `npos`）、块数据不完整（`buffer.size() - pos < chunk_size + 2`）、trailer 不完整。每种情况都返回 `complete = false` 和 `consumed`（已处理的字节数），调用者知道还需要等待更多数据。

问题2: "chunked 和 Content-Length 的区别？各自适用什么场景？"

答案：Content-Length 要求发送方**提前知道** body 的总字节数，适合静态文件（通过 `stat()` 获取文件大小）。Chunked 允许**边生成边发送**，适合动态内容（数据库查询、实时日志、API 流式输出）。从协议角度看，Content-Length 是一次性发送完整 body；chunked 是分块流式传输，用 `0\r\n\r\n` 标记结束。HTTP/1.1 中两者互斥——一个消息要么有 Content-Length，要么用 chunked 编码，不能同时使用（如果同时出现，chunked 优先）。

问题3: "Transfer-Encoding 和 Content-Encoding 的区别？"

答案：Transfer-Encoding 是**传输层**的编码，影响消息在网络上如何传输。它是一跳有效的（hop-by-hop）——代理可以修改或移除它。常见值：`chunked`。Content-Encoding 是**内容层**的编码，表示 body 的实际格式。它是端到端有效的（end-to-end）——代理不应该修改它。常见值：`gzip`、`deflate`、`br`。举例：服务器可以先对内容做 gzip 压缩（Content-Encoding: gzip），然后分块传输（Transfer-Encoding: chunked）。客户端先按 chunked 协议还原出完整的压缩数据，再按 gzip 解压得到原始内容。两者互不干扰。

第 4 课：Range 断点续传 + 304 条件请求

本节目标

实现 HTTP 缓存和断点续传的两个核心机制。完成后，你的服务器支持：

- 视频拖动（Range 请求 → 206 Partial Content）
- 大文件分段下载
- 浏览器缓存（304 Not Modified）
- ETag 缓存验证

理论知识

Range 请求：

客户端说"我只要文件的一部分"。三种格式：

Range: bytes=0-499	→ 前 500 字节 (0 到 499, 含两端)
Range: bytes=500-	→ 从第 500 字节到文件末尾
Range: bytes=-500	→ 最后 500 字节

服务器响应：

HTTP/1.1 206 Partial Content	
Content-Range: bytes 0-499/1048576	← 范围 / 总大小
Content-Length: 500	
Accept-Ranges: bytes	← 声明支持断点续传

条件请求：

客户端说"如果文件没变就不要发了，我用缓存"。两种验证方式：

If-Modified-Since: Thu, 12 Jun 2026 10:00:00 GMT	
→ 文件修改时间 ≤ 这个时间 → 304 Not Modified (无 body)	
→ 文件修改时间 > 这个时间 → 200 OK + 新内容	
If-None-Match: "abc123"	
→ ETag 匹配 → 304	
→ ETag 不匹配 → 200	

ETag 比时间更精确——时间精度只到秒，如果一秒内文件被改了两次，时间判断不出来，但 ETag 可以。

涉及文件

操作	文件	说明
新建	include/tiny_http/http_utils.h	日期/ETag 工具函数
新建	src/http_utils.cpp	工具函数实现
修改	src/linux_server.cpp	process_request() 增加 Range 和 304
修改	CMakeLists.txt	添加 http_utils.cpp

步骤 1：创建工具函数头文件

```
touch include/tiny_http/http_utils.h
vim include/tiny_http/http_utils.h
```

按 `i`，输入：

```
#pragma once

#include <string>
#include <ctime>

namespace tiny_http {

// 生成 RFC 1123 格式的 HTTP 日期 (如 "Thu, 12 Jun 2026 10:00:00 GMT")
// 注意: 必须是 GMT 时区!
std::string http_date(time_t t);

// 获取当前时间的 HTTP 日期格式
std::string http_date_now();

// 解析 HTTP 日期字符串为 time_t
// 支持 RFC 1123 格式 (最常用)
time_t parse_http_date(const std::string& date_str);

// 生成简单的 ETag
// 基于文件大小和修改时间组合
std::string generate_etag(size_t file_size, time_t mtime);

} // namespace tiny_http
```

按 `Esc`，输入 `:wq` 回车。

步骤 2：实现工具函数

```
touch src/http_utils.cpp
vim src/http_utils.cpp
```

按 **i**，输入：

```
#include "tiny_http/http_utils.h"

#include <sstream>
#include <iomanip>
#include <cstring>

namespace tiny_http {

std::string http_date(time_t t) {
    struct tm tm_buf;
    // gmtime_r 是线程安全版本的 gmtime
    // 必须用 GMT 时间，不能用本地时间！
    gmtime_r(&t, &tm_buf);

    char buf[64];
    // RFC 1123 格式: Sun, 06 Nov 1994 08:49:37 GMT
    strftime(buf, sizeof(buf), "%a, %d %b %Y %H:%M:%S GMT", &tm_buf);
    return std::string(buf);
}

std::string http_date_now() {
    return http_date(std::time(nullptr));
}

time_t parse_http_date(const std::string& date_str) {
    struct tm tm_buf = {};
    memset(&tm_buf, 0, sizeof(tm_buf));

    // 解析 RFC 1123 格式
    // strptime 不是 ISO C++ 标准，但在 Linux/BSD/macOS 上都可用
    char* rest = strptime(date_str.c_str(),
                          "%a, %d %b %Y %H:%M:%S", &tm_buf);
    if (rest != nullptr) {
        // timegm: 将 UTC 时间的 struct tm 转为 time_t
        // 注意: timegm 不是 POSIX 标准但在 Linux 上可用
        // 如果不可用，可以用 mktime 然后减去时区偏移
        return timegm(&tm_buf);
    }

    // 解析失败，返回 0 (epoch, 即 1970-01-01)
    return 0;
}

std::string generate_etag(size_t file_size, time_t mtime) {
    // 简单但有效的 ETag 生成:
    // "\文件大小-修改时间戳\"
    // 例如: "\1048576-1719000000\"
    std::ostringstream oss;
```

```

    oss << "\"" << std::hex << file_size
        << "-" << mtime << "\"";
    return oss.str();
}

} // namespace tiny_http

```

逐段解释：

- `http_date()`：用 `gmtime_r()` 把 `time_t` 转成 UTC 时间的 `struct tm`，再用 `strftime()` 格式化成 RFC 1123 格式。**必须用 GMT 时间**——HTTP 协议规定所有日期头都是 GMT。用 `localtime_r()` 会生成带时区的错误格式。
- `parse_http_date()`：用 `strptime()` 解析 RFC 1123 格式字符串（`%a` = 星期缩写、`%d` = 日期、`%b` = 月份缩写、`%Y` = 年份、`%H:%M:%S` = 时分秒）。再用 `timegm()` 转成 `time_t`。
- `generate_etag()`：组合文件大小（十六进制）和修改时间戳。例如大小 1048576、时间戳 1719000000 → "100000-1719000000"。引号是 HTTP 规范要求——ETag 值必须用双引号包裹。

按 `Esc`，输入 `:wq` 回车。

更新 CMakeLists.txt：

```
vim CMakeLists.txt
```

在 `add_library(tiny_http_core` 的源文件列表中添加 `src/http_utils.cpp`。按 `i`，在 `src/config_parser.cpp` 下面加一行：

```
src/http_utils.cpp
```

按 `Esc`，输入 `:wq` 回车。

验证编译：

```
cmake -S . -B build && cmake --build build
```

期望编译成功。

步骤 3：修改 `linux_server.cpp` —— 增加 Range 和 304 支持

这是本课最核心的改动。需要修改 `process_request()` 函数。

```
vim src/linux_server.cpp
```

首先，在文件顶部的 `include` 区域新增两个头文件：

按 i，在 `#include <sstream>` 下面（或 `#include <fstream>` 附近）新增：

```
#include <sys/stat.h>          // stat() 获取文件信息
#include "tiny_http/http_utils.h" // 日期和 ETag 工具
```

然后，找到 `process_request()` 函数中"打开文件"的部分（约第 333 行的 `std::ifstream file(path, std::ios::binary);`）。

我们需要把这一段逻辑重写。原来的流程是：

```
ifstream 打开文件 → 读全部内容 → 生成响应
```

新的流程是：

```
stat() 获取文件信息 → 检查 304 → 检查 Range →
如果是 Range: 读指定范围 → 206
否则: 正常读全部 → 200
```

找到 `process_request()` 中从 `std::ifstream file(...)` 开始到函数结束的部分，将其替换为以下完整逻辑：

```
// ===== 第4步: 用 stat() 获取文件信息 =====
struct stat file_stat;
if (::stat(path.c_str(), &file_stat) != 0) {
    // stat 失败 (文件不存在或无权限)
    HttpResponse response(404, "Not Found");
    response.set_body("Not Found");
    response.set_keep_alive(!conn.close_after_write);
    conn.out = response.serialize();
    switch_to_epollout(fd);
    return;
}

// 拒绝目录请求
if (S_ISDIR(file_stat.st_mode)) {
    HttpResponse response(403, "Forbidden");
    response.set_body("Forbidden");
    response.set_keep_alive(!conn.close_after_write);
    conn.out = response.serialize();
    switch_to_epollout(fd);
    return;
}

size_t file_size = static_cast<size_t>(file_stat.st_size);
time_t file_mtime = file_stat.st_mtime;

// ===== 第5步: 条件请求检查 (304) =====
// 检查 If-Modified-Since
const std::string if_mod = request.header("If-Modified-Since");
if (!if_mod.empty()) {
    time_t client_time = parse_http_date(if_mod);
```

```

        if (client_time > 0 && file_mtime <= client_time) {
            // 文件未修改 → 304
            HttpResponse response(304, "Not Modified");
            response.set_header("Last-Modified", http_date(file_mtime));
            response.set_header("ETag", generate_etag(file_size, file_mtime));
            response.set_header("Cache-Control", "no-cache");
            response.set_keep_alive(!conn.close_after_write);
            conn.out = response.serialize();
            switch_to_epollout(fd);
            return;
        }
    }

    // 检查 If-None-Match
    const std::string if_none = request.header("If-None-Match");
    if (!if_none.empty()) {
        std::string etag = generate_etag(file_size, file_mtime);
        if (if_none == etag) {
            // ETag 匹配 → 304
            HttpResponse response(304, "Not Modified");
            response.set_header("ETag", etag);
            response.set_header("Last-Modified", http_date(file_mtime));
            response.set_header("Cache-Control", "no-cache");
            response.set_keep_alive(!conn.close_after_write);
            conn.out = response.serialize();
            switch_to_epollout(fd);
            return;
        }
    }

    // ===== 第6步: 检查 Range 请求 (206) =====
    bool is_range = false;
    size_t range_start = 0;
    size_t range_end = file_size - 1; // 默认到末尾

    const std::string range_header = request.header("Range");
    if (!range_header.empty() && request.method == "GET") {
        const std::string prefix = "bytes=";
        if (range_header.size() > prefix.size()
            && range_header.substr(0, prefix.size()) == prefix) {

            std::string range_value = range_header.substr(prefix.size());
            size_t dash = range_value.find('-');

            if (dash != std::string::npos) {
                try {
                    std::string start_str = range_value.substr(0, dash);
                    std::string end_str = range_value.substr(dash + 1);

                    if (!start_str.empty() && !end_str.empty()) {
                        // "bytes=0-499"
                        range_start = std::stoul(start_str);
                        range_end = std::stoul(end_str);
                    } else if (!start_str.empty()) {
                        // "bytes=500-" → 从 500 到末尾
                        range_start = std::stoul(start_str);
                        range_end = file_size - 1;
                    } else if (!end_str.empty()) {

```

```

        // "bytes=-500" → 最后 500 字节
        size_t suffix = std::stoul(end_str);
        if (suffix > file_size) suffix = file_size;
        range_start = file_size - suffix;
        range_end = file_size - 1;
    }

    // 有效性检查
    if (range_start <= range_end
        && range_end < file_size) {
        is_range = true;
    }
} catch (...) {
    // 解析失败, 忽略 Range 头, 正常返回 200
}

}

// ===== 第7步: 打开并读取文件 =====
// 注意: 这里仍然使用 ifstream 读取,
// 第5课会用 open()+sendfile() 替代以获得零拷贝
std::ifstream file(path, std::ios::binary);
if (!file) {
    HttpResponse response(404, "Not Found");
    response.set_body("Not Found");
    response.set_keep_alive(!conn.close_after_write);
    conn.out = response.serialize();
    switch_to_epollout(fd);
    return;
}

std::string body;
if (is_range) {
    // 读取指定范围
    size_t content_length = range_end - range_start + 1;
    body.resize(content_length);
    file.seekg(static_cast<std::streamoff>(range_start));
    file.read(&body[0], static_cast<std::streamoff>(content_length));
    body.resize(static_cast<size_t>(file.gcount())); // 实际读取的字节数
} else {
    // 读取整个文件
    std::ostringstream contents;
    contents << file.rdbuf();
    body = contents.str();
}
file.close();

// ===== 第8步: 生成响应 =====
if (is_range) {
    HttpResponse response(206, "Partial Content");
    response.set_body(
        request.method == "HEAD" ? std::string{} : body,
        mime_type_for_path(path));
    response.set_header("Content-Range",
        "bytes " + std::to_string(range_start) + "-"
        + std::to_string(range_end) + "/"
        + std::to_string(file_size));
}

```

```

        response.set_header("Accept-Ranges", "bytes");
        response.set_header("ETag", generate_etag(file_size, file_mtime));
        response.set_header("Last-Modified", http_date(file_mtime));
        response.set_keep_alive(!conn.close_after_write);
        conn.out = response.serialize();
    } else {
        HttpResponse response(200, "OK");
        response.set_body(
            request.method == "HEAD" ? std::string{} : body,
            mime_type_for_path(path));
        response.set_header("Accept-Ranges", "bytes");
        response.set_header("ETag", generate_etag(file_size, file_mtime));
        response.set_header("Last-Modified", http_date(file_mtime));
        response.set_keep_alive(!conn.close_after_write);
        conn.out = response.serialize();
    }

    switch_to_epollout(fd);

```

关键变更总结：

- 用 `stat()` 替代了原来隐式的 `ifstream` 错误检查（原来用 `if (!file)` 判断文件是否存在，现在提前用 `stat()` 获取元数据）
- 在文件读取之前插入 304 检查（优先于 Range——如果客户端缓存是最新的，直接返回 304，省去读文件的开销）
- 在文件读取之前解析 Range 头，确定要读取的范围
- 所有响应都加了 `ETag`、`Last-Modified`、`Accept-Ranges` 头

按 `Esc`，输入 `:wq` 回车。

步骤 4：编译并验证

```
cmake --build build
```

测试1：Range 请求：

```

# 终端1: ./build/tiny_httpd
# 终端2:

# 取前 100 字节
curl -v -H "Range: bytes=0-99" http://localhost:8080/test.html
# 期望: HTTP/1.1 206 Partial Content
#       Content-Range: bytes 0-99/xxx

# 取最后 100 字节
curl -v -H "Range: bytes=-100" http://localhost:8080/test.html

```

```
# 从第 50 字节到末尾
curl -v -H "Range: bytes=50-" http://localhost:8080/test.html
```

测试2：304 Not Modified:

```
# 先获取 Last-Modified 时间
curl -v http://localhost:8080/test.html 2>&1 | grep -i "Last-Modified"

# 用获取到的时间发条件请求
curl -v -H 'If-Modified-Since: Thu, 12 Jun 2026 10:00:00 GMT' \
    http://localhost:8080/test.html
# 期望: HTTP/1.1 304 Not Modified (无 body)
```

测试3：确认原有功能未破坏:

```
curl -v http://localhost:8080/
# 期望: 200 OK, 返回 index.html
./build/parser_tests
# 期望: parser tests passed
```

第 4 课练习答案

练习1：如果不检查 `range_start <= range_end` 就直接使用，攻击者能做什么？

答案：攻击者可以构造恶意 Range 头导致整数下溢或越界读取。例如：

- Range: bytes=100-0 : `range_start=100`, `range_end=0` , `content_length = 0-100+1` 会是一个巨大的正数（无符号整数回绕），导致尝试分配巨量内存或读取远超文件大小的数据。
- Range: bytes=999999999-999999999 : 如果文件只有 1000 字节，`range_end >= file_size`，会越界读取。

我们的代码做了两层检查：先检查 `range_start <= range_end` 和 `range_end < file_size`，再使用这些值。这确保了范围始终合法。

练习2：为什么 304 响应不应该有 body？

答案：304 的语义是“你缓存的内容还是最新的，继续用缓存”。既然客户端已经有一份完整的内容（在缓存里），服务器就不需要再传一次 body——传了也是浪费带宽。RFC 7232 明确规定 304 响应 MUST NOT 包含消息体。从实现角度，304 通常只包含缓存验证头（ETag、Last-Modified、Cache-Control），不包含 Content-Length 或 body。

练习3: ETag 用"文件大小-修改时间"组合有什么局限?

答案: 有三个主要局限:

1. **一秒内的多次修改**: 如果文件在 1 秒内被修改了两次 (比如快速连续写入), `mtime` 只记录到秒, 两次修改会有相同的 ETag。但实际上内容已经变了, 缓存应该失效。实际系统中文件系统的时间戳精度通常是秒级或毫秒级。
2. **多服务器部署**: 同一个文件部署在多台服务器上时, 不同服务器上文件的 inode 可能不同, `mtime` 也可能因为部署时间不同而有差异。这导致同一个 URL 在不同服务器上返回不同的 ETag, CDN 或负载均衡器无法用 ETag 做缓存去重。
3. **内容不变但 mtime 变了**: 如果用 `touch` 命令更新了文件的时间戳, 文件内容其实没变, 但 ETag 变了。客户端会被迫重新下载一份相同的内容。

更好的做法: 用文件内容的哈希值 (如 MD5 或 SHA1) 作为 ETag。这样 ETag 只依赖于文件的实际内容, 不受文件系统元数据影响。缺点是计算哈希需要读取整个文件, 对大文件开销较大。一个折中是只对小于某阈值 (如 1MB) 的文件计算哈希, 大文件继续用"大小-时间"组合。

第 4 课面试要点 (含答案)

问题1: "206 和 200 的区别? 什么时候返回 206?"

答案: 200 表示返回完整资源, body 包含整个文件内容。206 表示返回部分内容, body 只包含请求的范围。206 响应必须包含 `Content-Range` 头 (格式: `bytes start-end/total`), 告诉客户端"这是全部数据的哪一部分"。当客户端请求头包含有效的 `Range` 头时返回 206。此外, 服务器应先通过 `Accept-Ranges: bytes` 头声明自己支持断点续传。

问题2: "304 和 200 的区别? 什么条件触发 304?"

答案: 304 Not Modified 表示"你缓存的是最新的, 不需要重新下载"。触发条件: 客户端发送 `If-Modified-Since` (文件修改时间 $\leq$ 指定时间) 或 `If-None-Match` (ETag 匹配), 且服务器验证资源确实没有变化。304 不包含 body (省带宽), 只返回缓存验证头。200 返回完整的新内容。从性能角度看, 304 可以省去文件读取和网络传输的开销——只需要 `stat()` 一下文件就能返回响应。

问题3: "为什么用 `stat()` 而不是 `ifstream` 获取文件信息?"

答案: `std::ifstream` 是 C++ 标准库的文件输入流, 它封装了文件描述符但不暴露文件的 inode 元数据。它只能告诉你"文件能打开吗"和"读到了多少字节", 不能直接告诉你文件的大小 (除非 seek 到末尾再 tell)、修改时间、权限等。`stat()` 系统调用直接读取文件的 inode 信息, 一次调用就能拿到大小、修改时间、权限、类型 (普通文件/目录/符号链接)。对于 Range 请求 (需要文件总大小) 和条件请求 (需要修改时间), `stat()` 是必需的。

第 5 课：sendfile 零拷贝

本节目标

用 Linux 的 `sendfile()` 系统调用替代 `ifstream::read()` + `::send()` 组合，实现文件内容的零拷贝传输。这是本项目性能收益最大的单项优化——对大文件的吞吐量提升可达 2-4 倍。同时与第 4 课的 Range 功能整合，Range 请求也走零拷贝路径。

理论知识：零拷贝原理

当前数据路径（4 次上下文切换 + 2 次 CPU 拷贝）：

```
应用程序调用 read()      ← 上下文切换：用户态 → 内核态
内核：磁盘 → 页缓存      ← DMA 拷贝（非 CPU）
内核：页缓存 → 用户态 buffer ← 第1次 CPU 拷贝
应用程序的 read() 返回    ← 上下文切换：内核态 → 用户态

应用程序调用 send()      ← 上下文切换：用户态 → 内核态
内核：用户态 buffer → socket 缓冲区 ← 第2次 CPU 拷贝
内核：socket 缓冲区 → 网卡 ← DMA 拷贝（非 CPU）
应用程序的 send() 返回    ← 上下文切换：内核态 → 用户态
```

sendfile 数据路径（2 次上下文切换 + 0 次 CPU 拷贝）：

```
应用程序调用 sendfile()   ← 上下文切换：用户态 → 内核态
内核：磁盘 → 页缓存      ← DMA 拷贝
内核：页缓存 → socket 缓冲区 ← DMA 拷贝（如果网卡支持 scatter-gather，甚至可以跳过）
内核：socket 缓冲区 → 网卡 ← DMA 拷贝
应用程序的 sendfile() 返回 ← 上下文切换：内核态 → 用户态
```

数据从未进入用户态内存空间。这就是“零拷贝”——零 CPU 参与的数据复制。

sendfile 函数签名：

```
#include <sys/sendfile.h>

ssize_t sendfile(
    int out_fd,      // 目标文件描述符 (socket fd)
    int in_fd,       // 源文件描述符 (文件的 fd, 不是 ifstream!)
    off_t *offset,    // 从文件的哪个位置开始发送
                    // NULL = 从当前文件偏移开始
                    // 非 NULL = sendfile 会更新 *offset 为已发送的字节数
```

```
size_t count    // 发送多少字节
);
```

返回值：实际发送的字节数，0 表示 EOF，-1 表示错误（检查 `errno`）。

为什么小文件不用 `sendfile`：`sendfile` 有固定的系统调用开销和 DMA 设置成本。对于小于约 16KB 的文件，这个固定开销超过用户态拷贝的开销。所以我们设定一个阈值：小于 16KB 走原逻辑（`read+send`），大于 16KB 走 `sendfile`。

涉及文件

操作	文件	说明
修改	<code>include/tiny_http/linux_server.h</code>	Connection 增加 <code>sendfile</code> 字段
修改	<code>src/linux_server.cpp</code>	<code>process_request()</code> + <code>handle_write()</code> 重写

步骤 1：扩展 Connection 结构体

```
vim include/tiny_http/linux_server.h
```

按 `i` 进入编辑模式。找到 `struct Connection` 的定义，在最后一个字段 `long long last_active_ms {0};` 后面添加：

```
// ---- sendfile 零拷贝相关字段 ----
bool using_sendfile {false};           // 是否正在使用 sendfile 模式
int sendfile_fd {-1};                  // 打开的文件描述符（不是 ifstream）
off_t sendfile_offset {0};             // 当前发送偏移量
size_t sendfile_remaining {0};         // 剩余要发送的字节数
```

逐字段解释：

- `using_sendfile`：标志位。为 `true` 时 `handle_write()` 走 `sendfile` 分支。
- `sendfile_fd`：用 `open()` 打开的文件描述符。不能用 `ifstream` ——`sendfile` 需要原生的 `fd`。
- `sendfile_offset`：当前发送到文件哪个位置了。`sendfile` 每次调用后会自动更新这个值为“已发送的字节位置”，下次调用时从这个位置继续。
- `sendfile_remaining`：还剩多少字节要发。每次 `sendfile` 调用后减去实际发送的字节数，减到 0 表示发送完成。

按 `Esc`，输入 `:wq` 回车。

步骤 2: 修改 `handle_write()` —— 增加 `sendfile` 分支

```
vim src/linux_server.cpp
```

找到 `handle_write()` 函数。当前它只处理普通发送路径（`::send()` 发 `conn.out`）。我们需要在函数开头增加 `sendfile` 判断。

完整的 `handle_write()` 修改后版本:

```
void LinuxHttpServer::handle_write(int fd)
{
    auto it = connections_.find(fd);
    if (it == connections_.end()) {
        return;
    }
    Connection& conn = it->second;

    // =====
    // sendfile 路径 (新增)
    // =====
    if (conn.using_sendfile) {
        // 阶段1: 先发送响应头 (conn.out 中的内容)
        if (!conn.out.empty()) {
            ssize_t n = ::send(fd, conn.out.data(), conn.out.size(),
                               MSG_NOSIGNAL);
            if (n > 0) {
                conn.out.erase(0, static_cast<size_t>(n));
            } else if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
                return; // socket 发送缓冲区满了, 等下次 EPOLLOUT
            } else {
                // 发送错误
                if (conn.sendfile_fd >= 0) ::close(conn.sendfile_fd);
                close_connection(fd);
                return;
            }
        }
        // 如果 out 还没发完, 等下次 EPOLLOUT
        if (!conn.out.empty()) return;
    }

    // 阶段2: out 发完了 (或本来就是空的), 开始 sendfile
    while (conn.sendfile_remaining > 0) {
        ssize_t n = ::sendfile(
            fd, // socket fd (目标)
            conn.sendfile_fd, // 文件 fd (源)
            &conn.sendfile_offset, // 偏移量指针
            conn.sendfile_remaining // 要发送的字节数
        );
        if (n > 0) {
            conn.sendfile_remaining -= static_cast<size_t>(n);
            // sendfile_offset 已被 sendfile 自动更新
        } else if (n == 0) {
            // 文件读完了但剩余计数 > 0? 不应出现, 安全退出
            break;
        } else {
        }
```

```

        // n < 0
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            // socket 发送缓冲区满了，等下次 EPOLLOUT
            return;
        }
        // 真正的错误（如连接断开）
        break;
    }
}

// 阶段3: 清理
::close(conn.sendfile_fd);
conn.sendfile_fd = -1;
conn.using_sendfile = false;

if (conn.close_after_write) {
    close_connection(fd);
} else {
    switch_to_epollin(fd);
}
return;
}

// =====
// 普通发送路径（原逻辑，不变）
// =====
const std::string& chunk = conn.out;
const ssize_t n = ::send(fd, chunk.data(), chunk.size(), MSG_NOSIGNAL);

if (n > 0) {
    conn.out.erase(0, static_cast<size_t>(n));
    if (conn.out.empty()) {
        if (conn.close_after_write) {
            close_connection(fd);
        } else {
            switch_to_epollin(fd);
        }
    }
} else if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
    return;
} else {
    std::cerr << "send() fd=" << fd << " 错误 errno=" << errno << "\n";
    close_connection(fd);
}
}
}

```

sendfile 分支的三阶段设计：

1. **阶段1**：发响应头。如果 `conn.out` 非空，先把它发完。每次发一部分（因为是用非阻塞 socket），发不完整就 `return` 等下次 EPOLLOUT。
2. **阶段2**：out 空了，开始 while 循环调用 `sendfile`。每次调用可能只发一部分（socket 缓冲区有限），remaining 减少，offset 更新。循环直到 remaining 归零或遇到 EAGAIN。
3. **阶段3**：清理。关闭文件 fd，重置标志位，根据 Keep-Alive 决定关闭连接还是切回读模式。

步骤 3：修改 process_request() —— 增加 sendfile 发送路径

在同一个文件的 process_request() 函数中，重写文件读取和响应生成的部分。

找到当前的文件读取和响应生成代码（第 4 课改过的版本，约从 std::ifstream file(path, std::ios::binary); 开始到函数末尾），将其替换为：

```
// ===== 第7步：打开文件并生成响应 =====

// 小文件阈值：小于此值用 read+send，大于等于此值用 sendfile
constexpr size_t SMALL_FILE_THRESHOLD = 16384; // 16KB

// 确定要发送的内容大小
size_t send_size = is_range
    ? (range_end - range_start + 1)
    : file_size;

// ===== HEAD 请求：只发响应头 =====
if (request.method == "HEAD") {
    HttpResponse response(is_range ? 206 : 200,
        is_range ? "Partial Content" : "OK");
    response.set_header("Content-Type", mime_type_for_path(path));
    response.set_header("Content-Length", std::to_string(send_size));
    response.set_header("Last-Modified", http_date(file_mtime));
    response.set_header("ETag", generate_etag(file_size, file_mtime));
    response.set_header("Accept-Ranges", "bytes");
    if (is_range) {
        response.set_header("Content-Range",
            "bytes " + std::to_string(range_start) + "-"
            + std::to_string(range_end) + "/"
            + std::to_string(file_size));
    }
    response.set_keep_alive(!conn.close_after_write);
    conn.out = response.serialize();
    switch_to_epollout(fd);
    return;
}

// ===== 判断是否使用 sendfile =====
// 条件：文件足够大 + 不是 Range 小请求
bool use_sendfile = (send_size >= SMALL_FILE_THRESHOLD);

if (!use_sendfile) {
    // ---- 小文件路径：read + send（原逻辑） ----
    std::ifstream file(path, std::ios::binary);
    if (!file) {
        HttpResponse response(404, "Not Found");
        response.set_body("Not Found");
        response.set_keep_alive(!conn.close_after_write);
        conn.out = response.serialize();
        switch_to_epollout(fd);
        return;
    }

    std::string body;
    if (is_range) {
```

```

        body.resize(send_size);
        file.seekg(static_cast<std::streamoff>(range_start));
        file.read(&body[0], static_cast<std::streamoff>(send_size));
        body.resize(static_cast<size_t>(file.gcount()));
    } else {
        std::ostringstream contents;
        contents << file.rdbuf();
        body = contents.str();
    }
    file.close();

    HttpResponse response(is_range ? 206 : 200,
                          is_range ? "Partial Content" : "OK");
    response.set_body(std::move(body), mime_type_for_path(path));
    response.set_header("Last-Modified", http_date(file_mtime));
    response.set_header("ETag", generate_etag(file_size, file_mtime));
    response.set_header("Accept-Ranges", "bytes");
    if (is_range) {
        response.set_header("Content-Range",
                            "bytes " + std::to_string(range_start) + "-"
                            + std::to_string(range_end) + "/"
                            + std::to_string(file_size));
    }
    response.set_keep_alive(!conn.close_after_write);
    conn.out = response.serialize();
    switch_to_epollout(fd);
    return;
}

// ---- 大文件路径: sendfile 零拷贝 ----
// 用 open() 打开文件 (需要原生的 fd)
int file_fd = ::open(path.c_str(), O_RDONLY);
if (file_fd < 0) {
    HttpResponse response(500, "Internal Server Error");
    response.set_body("Internal Server Error");
    response.set_keep_alive(!conn.close_after_write);
    conn.out = response.serialize();
    switch_to_epollout(fd);
    return;
}

// 构建响应头 (不含 body), 用 serialize_headers()
HttpResponse response(is_range ? 206 : 200,
                      is_range ? "Partial Content" : "OK");
response.set_header("Content-Type", mime_type_for_path(path));
response.set_header("Content-Length", std::to_string(send_size));
response.set_header("Last-Modified", http_date(file_mtime));
response.set_header("ETag", generate_etag(file_size, file_mtime));
response.set_header("Accept-Ranges", "bytes");
if (is_range) {
    response.set_header("Content-Range",
                        "bytes " + std::to_string(range_start) + "-"
                        + std::to_string(range_end) + "/"
                        + std::to_string(file_size));
}
response.set_keep_alive(!conn.close_after_write);

// 只序列化头部 (body 由 sendfile 直接发送)

```

```

conn.out = response.serialize_headers();

// 设置 sendfile 参数
conn.using_sendfile = true;
conn.sendfile_fd = file_fd;
conn.sendfile_offset = static_cast<off_t>(is_range ? range_start : 0);
conn.sendfile_remaining = send_size;

switch_to_epollout(fd);
// 注意: file_fd 由 handle_write() 中的 sendfile 分支负责关闭

```

关键设计点:

- **16KB 阈值**: 小于此值的文件走原 read+send 路径（避免 sendfile 固定开销超过收益），大于等于此值走 sendfile 零拷贝。
- **HEAD 请求单独处理**: 只发响应头，不发 body。直接 `serialize()` 返回完整响应（含空 body）。
- **sendfile 路径不读取文件内容到用户态**: 只构建响应头（`serialize_headers()`），文件 body 由 `handle_write()` 中的 sendfile 直接从内核发送。
- **Range + sendfile 整合**: `sendfile_offset` 直接设为 Range 的起始位置，`sendfile_remaining` 设为范围大小。一行额外代码都不用。

步骤 4: 防止文件描述符泄漏

修改 `close_connection()` 函数，确保在关闭连接时也关闭可能打开的 sendfile 文件描述符:

```

void LinuxHttpServer::close_connection(int fd)
{
    auto it = connections_.find(fd);
    if (it != connections_.end()) {
        // 如果这个连接正在用 sendfile, 先关闭文件 fd
        if (it->second.sendfile_fd >= 0) {
            ::close(it->second.sendfile_fd);
            it->second.sendfile_fd = -1;
        }
    }

    // 从 epoll 移除
    ::epoll_ctl(epoll_fd_, EPOLL_CTL_DEL, fd, nullptr);
    // 关闭 socket fd
    ::close(fd);
    // 从连接表删除
    connections_.erase(fd);
    logger_.log(LogLevel::Info, "连接关闭 fd=" + std::to_string(fd)
        + " 剩余连接数=" + std::to_string(connections_.size()));
}

```

这是防御性代码：如果连接在 `sendfile` 中途被异常关闭（比如客户端断开），`handle_write()` 中的 `sendfile` 分支可能走不到"阶段3：清理"。`close_connection()` 作为最终的清理函数，必须检查并关闭文件 `fd`。

按 `Esc`，输入 `:wq` 回车。

同时需要在文件顶部添加 `sendfile` 的头文件。打开 `linux_server.cpp`：

```
vim src/linux_server.cpp
```

检查顶部是否已有 `#include <sys/sendfile.h>`。如果没有，在 `#include <sys/epoll.h>` 附近添加：

```
#include <sys/sendfile.h>    // sendfile() 系统调用
#include <sys/stat.h>        // stat() 获取文件信息（第4课已加）
```

步骤 5：编译并验证

```
cmake --build build
```

功能验证：

```
# 终端1: ./build/tiny_httpd
# 终端2:

# 测试1: 小文件（走原路径, <16KB）
curl -v http://localhost:8080/index.html
# 应该正常返回

# 测试2: 大文件（走 sendfile 路径）
curl -v http://localhost:8080/1m.bin -o /tmp/downloaded.bin
# 对比 md5 确认完整性
md5sum /tmp/downloaded.bin www/1m.bin
# 两个 md5 应该相同!

# 测试3: Range + sendfile 组合
curl -v -H "Range: bytes=100000-199999" http://localhost:8080/1m.bin -o /tmp/part.bin
ls -la /tmp/part.bin
# 应该是 100000 字节（200000-100000）

# 测试4: 小 Range（走原路径, <16KB）
curl -v -H "Range: bytes=0-999" http://localhost:8080/1m.bin
# 应该正常返回 206
```

性能对比：

```
# 运行第2课创建的压测脚本
./benchmarks/run_benchmark.sh

# 对比这次的结果和 baseline_result.txt
echo "=== 优化前（基线）==="
grep "Requests/sec\|Transfer/sec" benchmarks/baseline_result.txt
echo ""
echo "=== 优化后（sendfile）==="
grep "Requests/sec\|Transfer/sec" benchmarks/results_$(ls -t benchmarks/results_* |
head -1 | cut -d_ -f2-)/*.txt
```

重点关注 1MB 文件的 Transfer/sec——期望提升 2-4 倍。

第 5 课练习答案

练习1：为什么对小文件（<16KB）不用 sendfile？

答案：sendfile 有固定的系统调用开销——进入内核态、设置 DMA 描述符、配置 scatter-gather 列表、返回用户态。对于 1KB 的小文件，这个固定成本（可能几微秒）可能比“把 1KB 数据从内核拷贝到用户态再发出去”还大。另外，sendfile 需要 `open()` + `close()` 文件描述符，而小文件用 `ifstream` 的 RAII 自动管理更简洁。16KB 这个阈值是经验值，你可以根据自己的硬件测试调整。

练习2：sendfile() 的 offset 参数为什么是指针而不是值？

答案：因为 sendfile 需要告诉调用者实际发送了多少字节。sendfile 和 write/send 一样，可能返回“短写入”——比如你请求发送 1MB，但 socket 发送缓冲区只有 64KB 空间，sendfile 只会发送 64KB，然后返回 65536。下次你再次调用 sendfile 时，需要从“上次中断的地方”继续发送。offset 指针就让内核帮你记录这个位置——每次 sendfile 返回后，`*offset` 的值就是“已成功发送到的文件位置”。如果你传 NULL，sendfile 会使用文件当前的偏移量（lseek 位置），但多线程环境下这不可靠。我们显式传指针，每次循环后 offset 自动更新，remaining 手动减少。

练习3：如果客户端通过 HTTPS 连接，sendfile 还能用吗？为什么？

答案：不能直接使用。HTTPS 在 TCP 之上有一层 TLS 加密——数据必须在发送前被加密。sendfile 直接从内核页缓存取原始数据写入 socket 缓冲区，绕过了用户态的 TLS 加密逻辑。如果对 HTTPS 连接使用 sendfile，客户端收到的将是**未加密的明文数据**，TLS 协议会立即检测到并断开连接。

解决方案：Linux 内核从 4.13 开始支持 **kTLS**（Kernel TLS）——TLS 加密可以在内核中完成。启用 kTLS 后，可以将加密密钥设置到 socket 上，然后 sendfile 发送的数据会在内核中自动加密。但这需要：

1. Linux 内核 $\geq$ 4.13

2. 应用程序通过 `setsockopt(TCP_ULP, "tls")` 启用 kTLS

3. 手动协商 TLS 握手并将密钥传递给内核

对于学习项目来说，我们只需要知道"HTTP 用 `sendfile`，HTTPS 不能用"这个区别就够了。

第 5 课面试要点（含答案）

问题1: "零拷贝到底是什么？说清楚数据是怎么走的。"

答案：零拷贝是指数据从磁盘到网络的过程中，**不经过用户态内存空间**，CPU 不参与数据复制。传统方式：磁盘→内核页缓存→（CPU拷贝）→用户态buffer→（CPU拷贝）→socket缓冲区→网卡，需要 4 次上下文切换和 2 次 CPU 拷贝。sendfile 方式：磁盘→内核页缓存→（DMA拷贝）→socket缓冲区→网卡，只需要 2 次上下文切换和 0 次 CPU 拷贝。数据路径全程在内核空间完成。

实际测试中，1MB 文件传输的吞吐量从约 200MB/s 提升到约 800MB/s（在 SSD 和千兆网卡环境下）。这个数字因硬件而异，但 2-4 倍的提升是典型的。小文件（<16KB）收益不明显，因为 sendfile 的固定开销（DMA 设置、系统调用）抵消了零拷贝的优势。

问题2: "sendfile 和 mmap+write 的区别？各有什么优缺点？"

答案：mmap 把文件映射到进程地址空间（内核页缓存映射到用户态虚拟地址），然后 write 发送。数据路径：磁盘→页缓存→（映射，非拷贝）→用户态可见→（CPU拷贝到socket缓冲区）→网卡。只有 1 次 CPU 拷贝（比传统方式的 2 次少），但不是零拷贝。sendfile 是 0 次 CPU 拷贝。

mmap 的优势：更灵活——你可以修改映射区域的数据再发送（比如加水印、格式转换）。sendfile 只能原样发送，不能修改。mmap 的劣势：映射和解除映射有开销（页表操作），处理大文件时可能导致 TLB 抖动。

选择建议：纯文件传输（静态文件服务器）用 sendfile；需要修改数据再发送（动态内容、模板渲染）用 mmap 或传统 read/write。

问题3: "你实际测出来性能提升了多少？怎么测试的？"

答案：（准备好你的具体数字）用 wrk 测试 1MB 文件的吞吐量。测试环境：4 线程、10 并发连接、持续 30 秒。优化前 Transfer/sec 约 X MB/s，优化后约 Y MB/s，提升约 Z 倍。小文件（1B）的 QPS 在优化前后基本不变（因为瓶颈不在文件 I/O，而在事件循环和线程调度），大文件的吞吐量提升最为显著。这是我完整记录了性能数据的测试脚本 `run_benchmark.sh`（展示脚本内容）——每次优化都有时间戳的结果目录，可以追溯完整的性能演进历史。

问题4: "sendfile 有什么限制? "

答案: 四个主要限制:

1. **源必须是文件描述符** (支持 mmap 的文件系统, 普通文件 OK, 管道和终端不行)
2. **目标必须是 socket 描述符** (Linux 特有——BSD 的 sendfile 签名不同)
3. **只能单向传输** (文件→socket, 不能反过来)
4. **不能用于加密连接** (HTTPS 需要用户态加密, 除非启用 kTLS)

另外 sendfile 在 Linux 2.6.33 之前只支持 TCP socket, 之后才支持任何 socket 类型。

第二部分: 剩余升级路线

以下按难度和依赖排序。每个课时包含核心思路、关键文件、难度评估。完成前 5 个核心课时后, 按这个顺序逐步攻克。

第 6 课：POST / PUT / DELETE 方法

难度：★★☆☆☆ | 预计时间：3-5 小时

核心思路：在 `process_request()` 中将方法白名单从 `GET/HEAD` 扩展为：

方法	行为	安全注意
POST	读取 body，保存到 <code>www/uploads/<时间戳>.dat</code>	限制 body 大小（用 <code>config.max_body_size</code> ）
PUT	读取 body，覆盖写入 path 对应的文件	必须检查路径穿越！限制文件大小
DELETE	调用 <code>unlink()</code> 删除 path 对应的文件	必须检查路径穿越！禁止删除目录
OPTIONS	返回 <code>Allow: GET, HEAD, POST, PUT, DELETE, OPTIONS</code>	简单实现即可

关键文件：[linux_server.cpp:311](#) 的方法检查处。

依赖：HTTP 解析器（已支持 body 解析）、配置系统（`max_body_size`）。

第 7 课：301 / 302 重定向

难度：★★☆☆☆ | **预计时间：**2-3 小时

核心思路：在配置文件中支持 `location` 规则的新指令 `return 301 <url>;`。解析后存到 `LocationRule`。在 `process_request()` 中匹配后返回：

```
HTTP/1.1 301 Moved Permanently
Location: /new-path/
```

301（永久）和 302（临时）的区别：301 浏览器会缓存重定向，下次直接访问新地址；302 每次都会询问服务器。

关键文件：`config_parser.cpp`（新增指令）、`linux_server.cpp`（处理重定向）。

第 8 课：请求体限制 + 速率限制 + 连接数限制

难度：★★☆☆☆ | 预计时间：3-4 小时

三个子功能：

1. **请求体大小限制**：解析器中检查 `Content-Length > max_body_size` → 返回 413 Payload Too Large
2. **速率限制**：基于 IP 的简单令牌桶。用 `std::unordered_map<std::string, int>` 记录每个 IP 的请求计数，每秒重置。超过 `config.rate_limit` → 返回 429 Too Many Requests
3. **每 IP 最大连接数**：在 `accept_clients()` 中统计每 IP 的连接数，超限拒绝新连接（直接 `close client_fd`）

关键文件：`http_parser.cpp`（body 大小检查）、`linux_server.cpp`（速率和连接数限制）。

第 9 课：对象池与内存池

难度：★★★☆☆ | 预计时间：5-8 小时

核心思路：预先分配对象和 buffer，避免运行时的动态内存分配。

```
// 对象池：预分配 N 个 Connection 对象
template<typename T, size_t N>
class ObjectPool {
    std::array<T, N> slots_;
    std::vector<size_t> free_list_;    // 空闲 slot 索引
public:
    T* acquire();    // 从 free_list 取一个
    void release(T*); // 归还索引到 free_list
};

// Buffer 池：预分配固定大小读写 buffer
class BufferPool {
    std::vector<char*> free_buffers_;
    size_t buffer_size_; // 如 8192
public:
    char* acquire();
    void release(char*);
};
```

收益：消除高并发下的内存分配/释放抖动。malloc/free 在多线程环境有全局锁竞争。

关键文件：新增 `include/tiny_http/object_pool.h`，修改 `linux_server.cpp` 使用对象池管理 Connection。

第 10 课：时间轮定时器

难度：★★★☆☆ | 预计时间：5-8 小时

当前问题：`sweep_idle_connections()` 每 100ms 遍历整个 `connections_map`， $O(n)$ 复杂度。10000 连接时每轮扫描 10000 个。

时间轮方案：将时间分成固定大小的槽（如每槽 1 秒），每个槽存储在该秒到期的连接链表。

```
slot[0] → conn{A, expire=0s} → conn{B, expire=0s}
slot[1] → conn{C, expire=1s}
slot[2] → (空)
...
slot[7] → (空)
↑ 指针每 1 秒步进一格，当前槽中的连接全部超时
```

操作	原实现	时间轮
插入	$O(1)$	$O(1)$
更新活跃时间	$O(1)$	$O(1)$ （移到新槽）
超时检测	$O(n)$	$O(1)$ （指针步进）

关键文件：新增 `include/tiny_http/timer_wheel.h`，替换 `sweep_idle_connections()`。

第 11 课：I/O 多路复用抽象层（Reactor 模式）

难度：★★★★☆☆ | 预计时间：6-10 小时

核心思路：抽象出 `Reactor` 接口，让服务器不依赖具体的 I/O 多路复用机制。

```
struct PollEvent {
    int fd;
    uint32_t events; // 可读、可写、错误
};

class Reactor {
public:
    virtual ~Reactor() = default;
    virtual void add(int fd, uint32_t events) = 0;
    virtual void mod(int fd, uint32_t events) = 0;
    virtual void del(int fd) = 0;
    virtual int wait(std::vector<PollEvent>& events, int timeout_ms) = 0;
};
```

- `EpollReactor`：封装现有 `epoll` 代码（Linux）
- `KqueueReactor`：macOS/BSD 的实现（预留，接口相同）

收益：一次抽象，解锁跨平台。代码改动限于 `LinuxHttpServer` 的构造函数和事件循环。

关键文件：新增 `include/tiny_http/reactor.h`，新增 `src/epoll_reactor.cpp`，修改 `linux_server.cpp`。

第 12 课：FastCGI 支持

难度：★★★★☆ | 预计时间：10-15 小时

核心思路：FastCGI 是 Web 服务器与 PHP/Python 等后端通信的二进制协议。

客户端 → tiny_httpd → (FastCGI 二进制协议, Unix socket) → PHP-FPM → 响应返回

FastCGI 记录结构（简化）：

```
struct FCGI_Header {
    uint8_t version;           // = 1
    uint8_t type;              // 请求类型
    uint16_t request_id;       // 请求 ID（支持多路复用）
    uint16_t content_length;    // body 长度
    uint8_t padding_length;    // 填充长度
    uint8_t reserved;          // 保留
};
```

实现要点：封装 FastCGI 消息、连接 PHP-FPM、将 HTTP 请求参数转为 FastCGI 参数（SCRIPT_FILENAME、REQUEST_METHOD 等）、读取 FastCGI 响应并转回 HTTP 响应。

参考：FastCGI 规范 https://fastcgi-archives.github.io/FastCGI_Specification.html

关键文件：新增 `src/fastcgi_client.cpp`，修改 `linux_server.cpp` 增加 FastCGI 转发。

第 13 课：反向代理

难度：★★★★☆ | 预计时间：10-15 小时

核心思路：HTTP/1.1 反向代理——将请求转发到后端 HTTP 服务器。

客户端 → tiny_httpd → (HTTP/1.1 转发) → 后端服务(如 Flask on :5000) → 响应透传

实现要点：

1. 根据 location 规则匹配 `proxy_pass`
2. 创建到后端的 TCP 连接（或从连接池复用 Keep-Alive 连接）
3. 修改 Host 头为后端地址
4. 透传请求头和 body 到后端
5. 读取后端响应，透传响应头和 body 回客户端
6. 连接池管理（避免每次请求都新建连接）

关键文件：新增 `src/reverse_proxy.cpp`，修改 `linux_server.cpp`。

第 14 课: WebSocket 协议升级

难度: ★★★★★☆ | 预计时间: 8-12 小时

核心思路: 通过 HTTP Upgrade 机制从 HTTP 升级到 WebSocket。

握手:

```
客户端请求:
GET /chat HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

服务器响应:
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
```

`Sec-WebSocket-Accept` 的计算: `BASE64(SHA1(客户端Key + "258EAF5E914-47DA-95CA-C5AB0DC85B11"))`

帧格式: 操作码 (文本/二进制/ping/pong/close)、掩码、载荷长度 (7/7+16/7+64 位可变)。

关键文件: 新增 `src/websocket.cpp`, 修改 `linux_server.cpp`。

第 15 课：TLS / HTTPS 集成

难度：★★★★★ | **预计时间：**15-20 小时

核心思路：集成 OpenSSL 实现 TLS 加密传输。

```
sudo apt install libssl-dev
```

```
#include <openssl/ssl.h>
#include <openssl/err.h>

// 初始化
SSL_library_init();
SSL_CTX* ctx = SSL_CTX_new(TLS_server_method());
SSL_CTX_use_certificate_file(ctx, "cert.pem", SSL_FILETYPE_PEM);
SSL_CTX_use_PrivateKey_file(ctx, "key.pem", SSL_FILETYPE_PEM);

// 每个连接
SSL* ssl = SSL_new(ctx);
SSL_set_fd(ssl, client_fd);
SSL_accept(ssl); // TLS 握手（阻塞！需要集成到 epoll）

// 读写（替代 recv/send）
SSL_read(ssl, buf, size);
SSL_write(ssl, buf, size);
```

难点： TLS 握手可能阻塞，需要集成到 `epoll` 事件循环（`ERR_WANT_READ/ERR_WANT_WRITE`）。`sendfile` 对 HTTPS 不可用（需要在用户态加密）。

关键文件： 新增 `src/tls_context.cpp`，修改 `linux_server.cpp`。

第 16 课：负载均衡

难度：★★★★★ | 预计时间：12-18 小时

核心思路：在第 13 课反向代理基础上增加后端池管理。

```
struct UpstreamBackend {
    std::string host;
    int port;
    int weight {1};
    bool healthy {true};
    int failures {0};
    int max_failures {3};
};

class LoadBalancer {
    std::vector<UpstreamBackend> backends_;
    size_t rr_index_{0}; // Round Robin 索引

    UpstreamBackend* pick_round_robin();
    UpstreamBackend* pick_least_conn();
    UpstreamBackend* pick_ip_hash(const std::string& client_ip);
    void health_check(); // 定期 HEAD /health 检查后端存活
};
```

三种算法：轮询（简单公平）、最少连接（长连接场景）、IP Hash（会话保持）。

关键文件：新增 `src/load_balancer.cpp`，修改 `linux_server.cpp`。

第 17 课：HTTP/2 支持

难度：★★★★★+ | 预计时间：30-50 小时

HTTP/2 是二进制帧协议，与 HTTP/1.1 文本协议完全不同：

特性	HTTP/1.1	HTTP/2
格式	文本，人类可读	二进制帧
连接	每请求串行（Keep-Alive 下复用）	一个连接多流并发（多路复用）
头部	纯文本，每次重复发送	HPACK 压缩
推送	不支持	服务器可主动推送资源

建议：作为远期目标。可以从 nghttp2 库入手简化实现，或者先理解协议再逐步手写。二进制帧解析、HPACK 压缩、流控制是三个核心难点。

关键文件：大幅改造或新增整个协议层。

附录 A：面试要点汇总

架构设计

- 1. 为什么用 epoll?** — epoll O(1) 事件注册，边缘触发模式减少系统调用次数，没有 fd 数量上限（select 默认 1024）。适合高并发场景。
- 2. ET 和 LT 的区别?** — ET（边缘触发）只在 fd 状态变化时通知一次，要求一次读到 EAGAIN；LT（电平触发）只要可读就持续通知。ET 减少 epoll_wait 调用次数，但编程复杂度更高——必须在一次通知中处理完所有数据。
- 3. 为什么线程池而不是每连接一线程?** — 线程创建/销毁开销大（约 100μs 创建 + 栈内存分配），高并发下线程数爆炸（10000 连接 = 10000 线程），上下文切换成为瓶颈。线程池固定线程数（如 4 个 = CPU 核心数），任务队列缓冲，避免线程数膨胀。
- 4. Keep-Alive 怎么实现?** — 响应发送完毕后不关闭连接，切回 EPOLLIN 等待该连接的下一个请求，同时启动超时定时器（config.keep_alive_seconds）。通过 HTTP 头 `Connection: keep-alive` 或 `close` 与客户端协商。

协议实现

5. 半包怎么处理？ — 解析器返回 Incomplete 状态，事件循环继续 `recv()` 追加数据到 `conn.in` 缓冲区，下次循环再调 `parse()` 重新解析。解析器不保存状态——每次从 `conn.in` 头开始解析，通过 `consumed` 字段删除已处理部分。

6. 路径穿越怎么防？ — 检测 `..` 和 `\` 字符（字符串级防护）。更底层可用 `realpath()` 获取规范路径后验证前缀是否在 `document_root` 内（系统级防护）。当前实现是最小化的字符串匹配版本。

7. `sendfile` 为什么快？ — 零拷贝：数据从磁盘页缓存直接 DMA 传输到 `socket` 缓冲区，不经过用户态。减少 2 次上下文切换和 2 次 CPU 拷贝。大文件吞吐量提升 2-4 倍。

8. `Chunked` vs `Content-Length`？ — `Content-Length` 需要提前知道 `body` 大小，适合静态文件；`Chunked` 可以边生成边发送，适合动态内容。两者在 HTTP/1.1 中互斥——同时出现时 `chunked` 优先（RFC 7230）。

并发与性能

9. BlockingQueue 怎么工作？ — mutex + condition_variable。push() 加锁入队 + notify_one() 唤醒一个等待线程；pop() 等待直到队列非空或关闭。close() 设置标志 + notify_all() 唤醒所有等待线程使它们退出。

10. 异步日志为什么快？ — 业务线程只做格式化 + 入队（持锁时间极短），不等待磁盘 I/O。后台线程批量刷盘（一次 fsync 写多条日志）。相比同步日志（每条日志都 fsync），异步日志的吞吐量高出 1-2 个数量级。

11. 优雅关闭怎么做？ — SIGINT/SIGTERM → 设置 running=false → 事件循环退出 → 线程池 close() 队列 → 等待所有工作线程 join() → 关闭所有客户端连接（epoll_ctl DEL + close fd）→ 关闭监听 socket → 最后停止日志线程（前面的操作还能记录日志）。

12. 怎么压测的？用什么工具？ — 用 wrk，分别测试 1B/1KB/1MB 文件，短连接和长连接两种模式，持续 30 秒。建立了自动化脚本 run_benchmark.sh，每次优化后跑一遍，按时间戳存档结果，形成完整的性能演进数据链。strace 做系统调用分析，定位瓶颈。

安全

13. 有哪些安全措施？ — 路径穿越防护（禁止 `..` 和 `\`）、请求头大小限制（8192 字节）、空闲连接超时清理（Keep-Alive timeout）、请求体大小限制（`config.max_body_size`）、目录请求拒绝（`S_ISDIR` 检查）。

14. 还没处理的安全风险？ — 慢速攻击（Slowloris：发送极慢的请求头耗尽连接）、请求走私（Request Smuggling：利用 Content-Length 和 Transfer-Encoding 的不一致处理）、HTTP 头注入（响应头中未过滤 `\r\n`）。这些都是后续可以加固的方向。

附录 B：推荐学习节奏

- 第 1-2 周： 第1课（配置文件）+ 第2课（压测基线）
 - 建立工程化基础，拿到性能基线数据
- 第 3-4 周： 第3课（Chunked 编码）
 - 深入理解 HTTP/1.1 协议，解析和生成两个方向都做
- 第 5-6 周： 第4课（Range + 304）
 - 补齐静态文件服务器的缓存和断点续传能力
- 第 7-8 周： 第5课（sendfile 零拷贝）
 - 性能优化，拿到优化前后的 QPS 对比数据
- 第 9-10 周： 第6课（POST/PUT/DELETE）+ 第7课（重定向）
 - 补齐 HTTP 方法能力
- 第 11-12 周： 第8课（安全加固）+ 第9课（内存池）
 - 安全 + 性能
- 第 13-14 周： 第10课（时间轮）+ 第11课（Reactor 抽象）
 - 架构优化，代码质量提升
- 第 15-18 周： 第12课（FastCGI）+ 第13课（反向代理）
 - 进军动态内容，从静态服务器升级为应用网关
- 第 19 周+： 第14-17课，按兴趣选择

附录 C：常用命令速查

```
# === 构建 ===
cmake -S . -B build && cmake --build build

# === 运行 ===
./build/tiny_httpd                # 默认配置
./build/tiny_httpd 9090 /var/www 8 # 命令行参数
./build/tiny_httpd -c my_config.conf # 指定配置文件

# === 测试 ===
./build/parser_tests                # 解析器测试
./build/config_parser_tests         # 配置解析器测试
curl -v http://localhost:8080/      # 基本请求
curl -v -H "Range: bytes=0-99" ...  # Range 请求
curl -v -H "If-Modified-Since: ..." # 条件请求
curl -v -X POST -H "Transfer-Encoding: chunked" --data-binary @- ...

# === 压测 ===
wrk -t4 -c100 -d30s http://localhost:8080/
./benchmarks/run_benchmark.sh      # 自动化压测脚本

# === 调试 ===
gdb ./build/tiny_httpd              # GDB 调试
strace -c -p $(pgrep tiny_httpd)     # 系统调用统计
perf record -p $(pgrep tiny_httpd) -g # CPU 采样
valgrind --tool=massif ./build/tiny_httpd # 内存分析

# === 日志 ===
tail -f tiny_httpd.log              # 实时日志
```

最后的话

这份指导书不是为了让你"看完"的——它是为了让你"做完"的。

每次打开一个课时：

1. 阅读"本节目标"和"理论知识"，理解要做什么
2. 按照"步骤"操作终端——打开文件、输入代码、保存、编译
3. 运行"验证"命令，确认功能正确
4. 阅读"练习答案"加深理解
5. 回顾"面试要点"——这些都是真实的面试问题，答案就是你自己写的代码

做完一个课时后：

- 运行 `./build/parser_tests` 确认没破坏已有功能
- 用 `curl` 手动测试各种边界情况
- 运行 `./benchmarks/run_benchmark.sh` 记录性能数据
- 可选：写一段简短的笔记记录踩过的坑

一个持续迭代的项目，比十个"做完就丢"的 demo 项目更有价值。

面试官在你的 commit 历史里，能看到一个工程师的真实成长轨迹——

从基础版 → 加配置系统 → 加压测 → 加协议支持 → 做性能优化 → 做架构升级。

祝你写代码愉快！